

Computer Vision 1

Convolutional Neural Networks

Dr. Martin R. Oswald

Lecture overview

Week		Date	Presenter	Lectures	Excerise / Labs
1	36	02/09/2024	Arnoud Visser	1. Introduction and Course Logistics; Camera model	
1	36	05/09/2024	Arnoud Visser	2. Sensor and image formation Color	
2	37	09/09/2024	Arnoud Visser	3. In place image processing Photometric stereo	
2	37	12/09/2024	Arnoud Visser	4. Convolution and image filtering	
3	38	16/09/2024	Arnoud Visser	5. Feature: edge and corner	
3	38	19/09/2024	Arnoud Visser	6. Motion and optical flow	
4	39	23/09/2024	Martin Oswald	7. Global transform, Image stitching, alignment	
4	39	26/09/2024	Martin Oswald	8. Basics of Recognition 1	Online Lectures
5	40	30/09/2024	Martin Oswald	9. Basics of Recognition 2	
5	40	03/10/2024	Martin Oswald	10. Convolutional neural networks (Basic)	
6	41	07/10/2024	Martin Oswald	11. Convolutional neural network (Training and testing)	
6	41	10/10/2024	Martin Oswald	12. Stereo, 3D reconstruction	
7	42	14/10/2024	TBA	13. Guest lectures	
7	42	17/10/2024	TBA; Martin & Arnoud	14. Guest lecture; Course Wrap-up, Exam Q&A	
8	43	21/10/2024	Your turn!	Final exam	

Today

Image Understanding

AN IMAGE IS WORTH A THOUSAND WORDS



Image Understanding

Classification



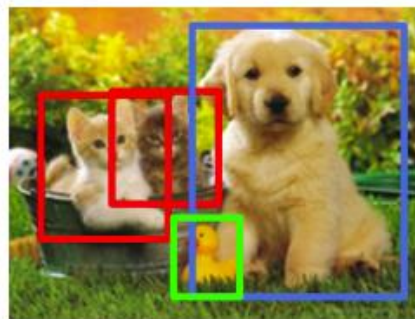
CAT

**Classification
+ Localization**



CAT

Object Detection



CAT, DOG, DUCK

**Instance
Segmentation**



CAT, DOG, DUCK

- Object Classification: classify the main object in an image if the image has one main object.
- Visual Image Retrieval: retrieve images from the library through a query image.
- Object Detection: identify whether there is an object in an image and recognize the object.
- Image Segmentation: segment an image into regions according the pixels the object belong to.

Recap

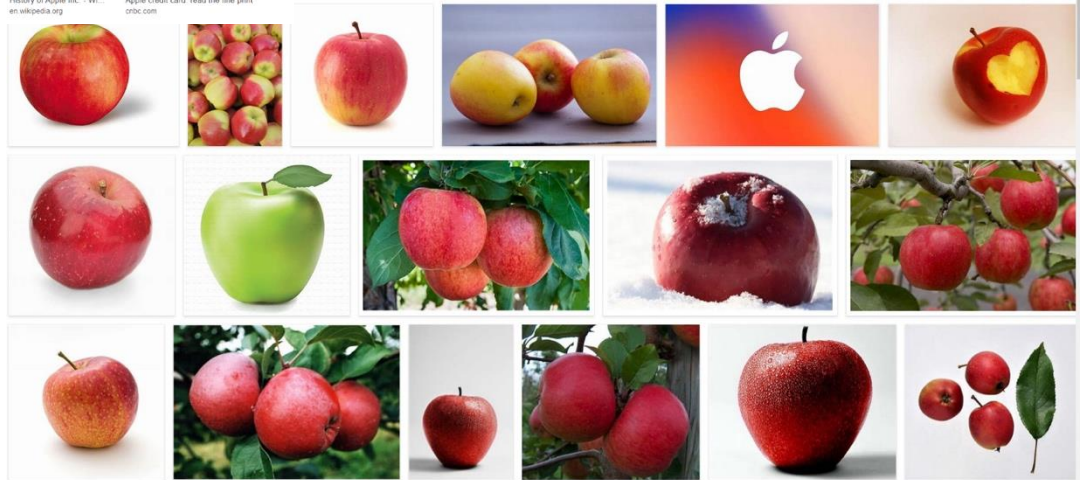
Image Retrieval

Search term: "appel"



Google Search

Bing Search



Retrieval Evaluation

Average Precision@K

- Consider rank position of each relevant images
 - $K_1, K_2, \dots, K_R$

- Compute Precision@K for each $K_1, K_2, \dots, K_R$

- Average precision = average of P@K

- Example:  has average precision:

rel: 1 0 1 0 1

$$AP@K = \frac{1}{3} * \left(\frac{1}{1} * 1 + \frac{1}{2} * 0 + \frac{2}{3} * 1 + \frac{2}{4} * 0 + \frac{3}{5} * 1 \right) \approx 0.76$$

relevance: {0,1}

↓

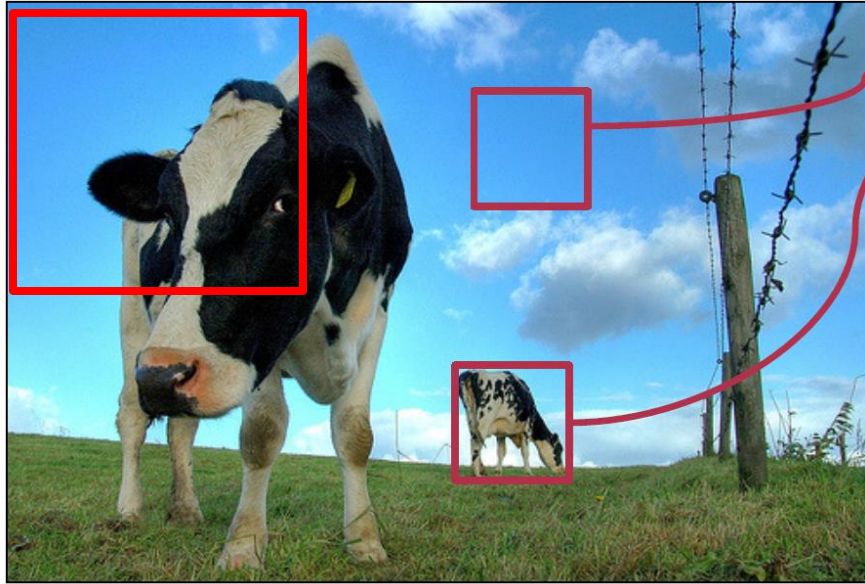
$$AP@K = \frac{\sum_{k=1}^K (P@k * rel_k)}{\# \text{ relevant results}}$$

Object Detection



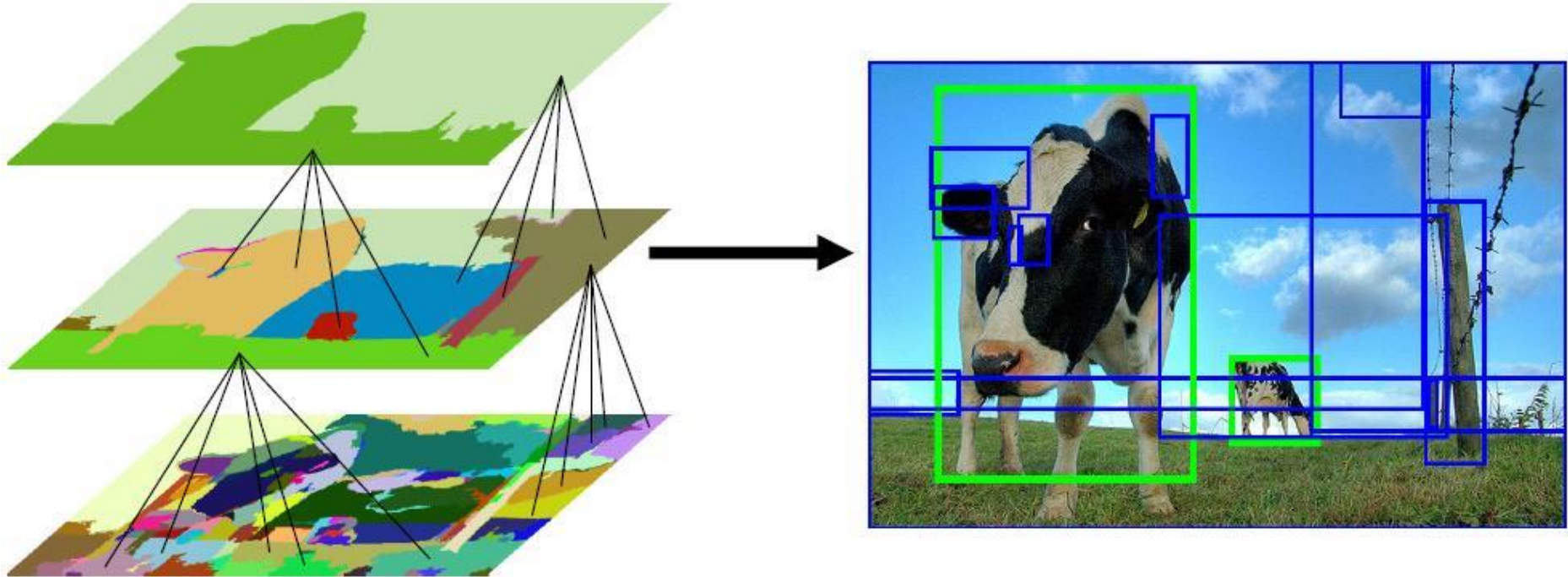
[<https://youtu.be/7BgiFrPs-x8?si>]

Region Proposals: Sliding Window



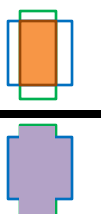
Is this a cow?

Region Proposals: Selective Search

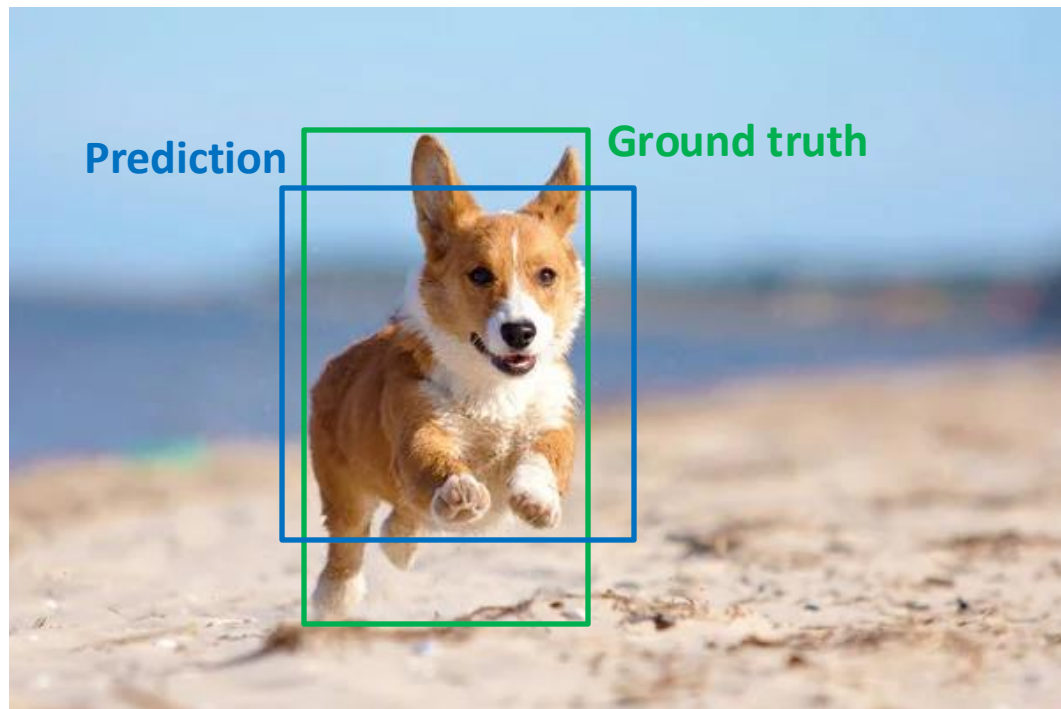


Comparing Bounding Boxes

- Intersection over Union (IoU)
(a.k.a. “Jaccard similarity/index”)

$$\text{IoU} = \frac{\text{Intersection}}{\text{Union}}$$


- Typically choose a threshold
e.g. $\text{IoU} \geq 0.5$, or 0.7
between prediction and GT
to decide about correct or
incorrect predictions



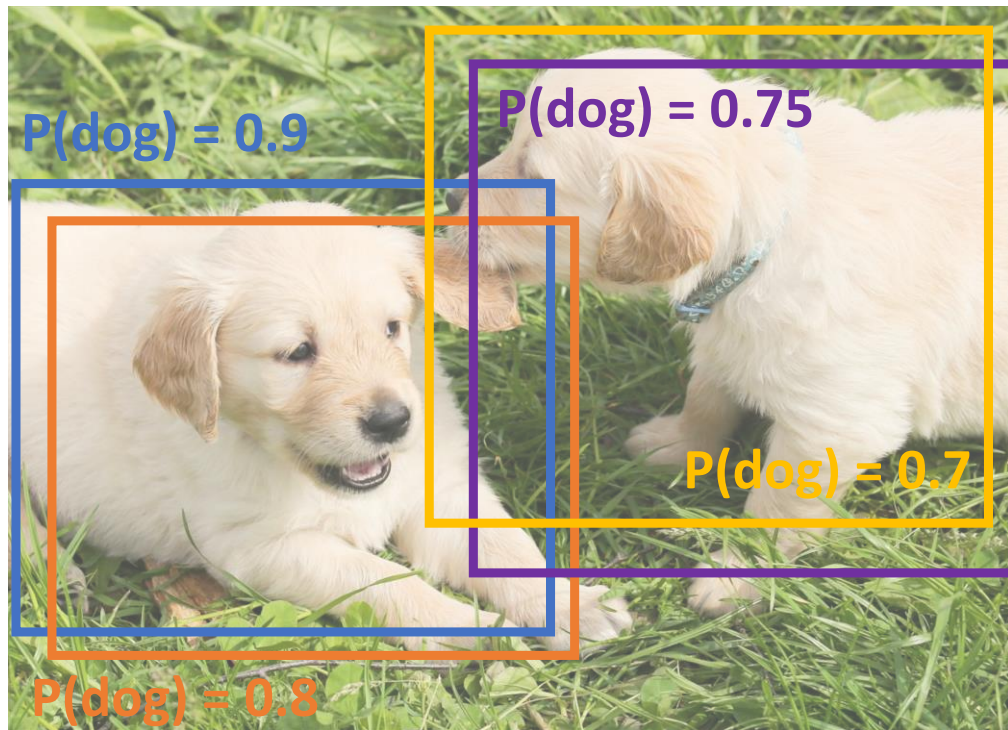
[<https://depositphotos.com/photos/corgi-running.html>]

Duplicate Filter: Non-Maximum Suppression (NMS)

Problem: Object detectors often output many overlapping detections

Solution: Post-process raw detections using **Non-Max Suppression (NMS)**:

1. Select next highest-scoring box
2. Eliminate lower-scoring boxes with $\text{IoU} > \text{threshold}$ (e.g. 0.7)
3. If any boxes remain, GOTO 1



[Puppy image is [CC0 Public Domain](#)]

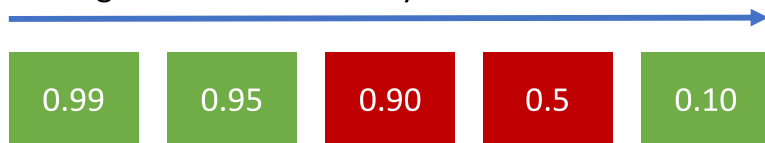
Evaluating Object Detectors

Mean Average Precision (mAP):

1. Run object detector on all test images (with NMS)
2. For each category, compute Average Precision (AP) = area under Precision vs Recall Curve
 1. For each detection (highest score to lowest score)
 1. If it matches some GT box with IoU > 0.5, mark it as positive and eliminate the GT
 2. Otherwise mark it as negative
 3. Plot a point on PR Curve

How to get AP = 1.0: Hit all GT boxes with IoU > 0.5, and have no “false positive” detections ranked above any “true positives”

All dog detections sorted by score



All ground-truth dog boxes

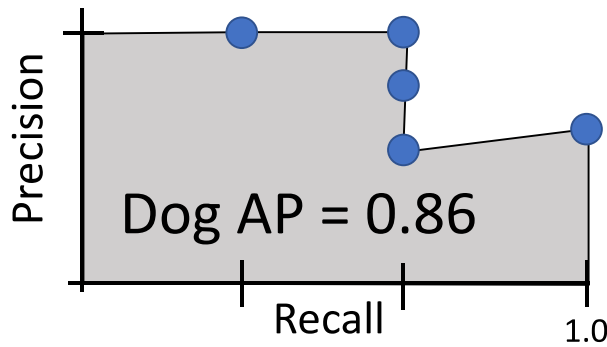


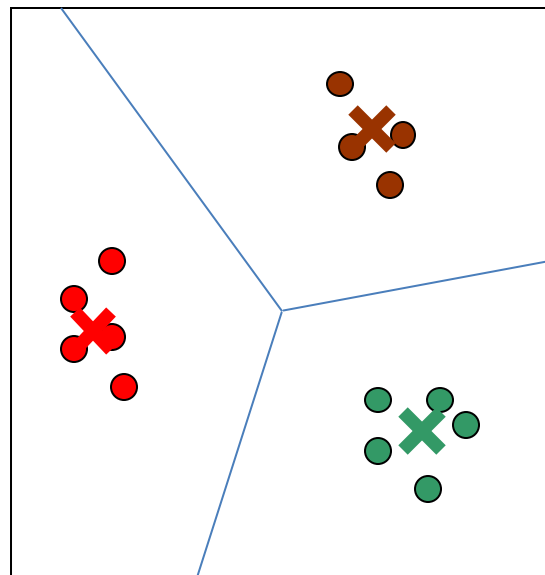
Image Segmentation

- Goal: Partition the image into semantically-meaningful or perceptually-similar regions.



K-Means

- Start with 3 random positions of the cluster centers.
- By computing the distance, *assign* each data point to the closest center.
- Re-compute the centers after the assignments.
- Iterate until no points are reassigned.



Iteration = 3

Today's Agenda



- **Convolutional Neural Networks**
- **Object Detection with Region-based CNNs**

Today's Agenda



- **Convolutional Neural Networks**
- Object Detection with Region-based CNNs

Historical Context

Traditional techniques till year 2000. Basis for neural networks already second half of last century.

1954 – (Harris) - Bag of Words

1972 – (Hough, Duda) Hough Transform

1943 – Neuron (McCulloch/Pitts)

1958 – Perceptron (Rosenblatt)

First AI Winter (1974-1980)

1986 – (Mc Connel) HoG

1986 – (Canny) – Canny Edge Detection

1988 – (Harris, Stephens), Harris Corner

1985 – BackPropagation (Werbos,
Parker, Rumelhart, Hinton,
Williams)

Second AI Winter (1987-1993)

1999 – (Lowe) SIFT

1992 – 1995 (Vapnik et al.) SVM

Historical Context

1989 - **LeNet-5** (by LeCun) – handwritten digit recognition

2010 - ImageNet large scale visual recognition challenge (ILSVRC)

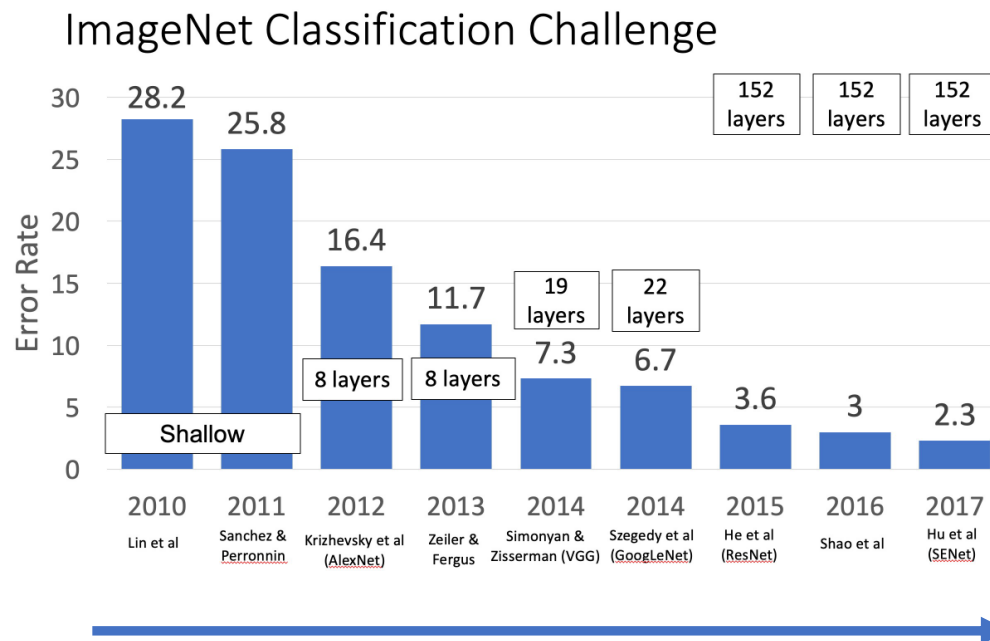
2012 - **AlexNet**

2014 - **VGGNet**

2015 - **ResNet**

2016 - **R-CNN**

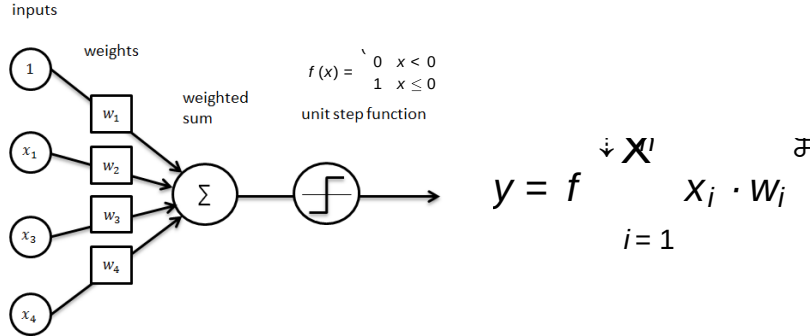
2018 - **Yolo - Darknet**



Artificial Neuron and Perceptron

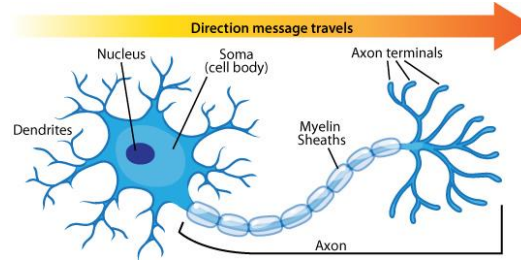
- Biology inspired McCulloch & Pitts (1943) to an artificial 'neuron'
- Rosenblatt added weights to the neuron model (1957)

Rosenblatt's Perceptron

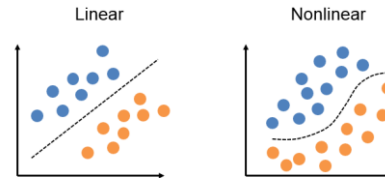
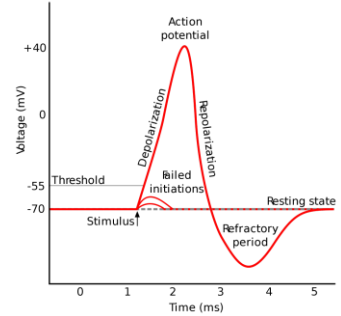


- Artificial neuron
- Linear classifier, does only work for linearly separable data

Biological model of a neuron

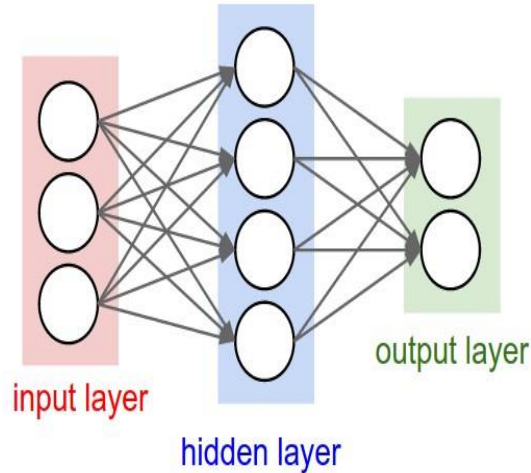


Stimulus behavior

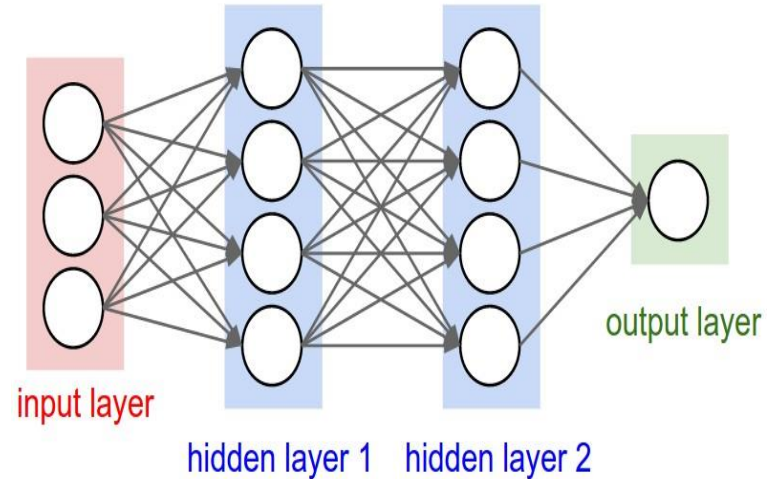


Multi-layer Perceptron (MLP)

- ...is a '*fully connected*' neural network with non-linear activation functions
- ...is a 'feed forward' neural network

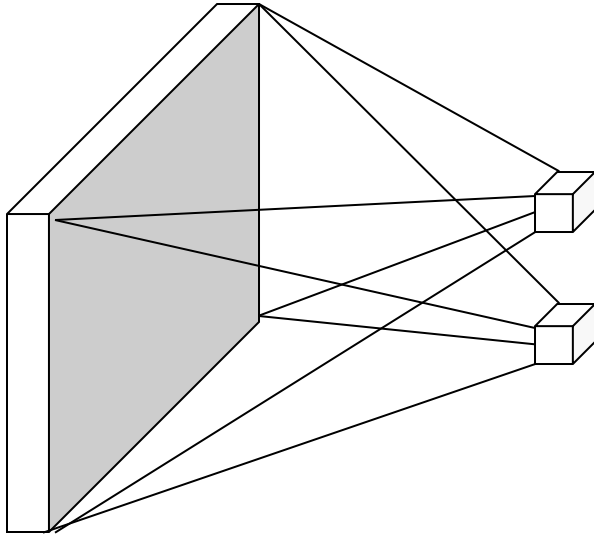


"2-layer Neural Net", or
"1-hidden-layer Neural Net"



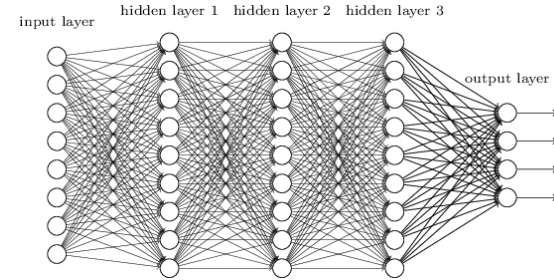
"3-layer Neural Net", or
"2-hidden-layer Neural Net"

A Neural Network for Images



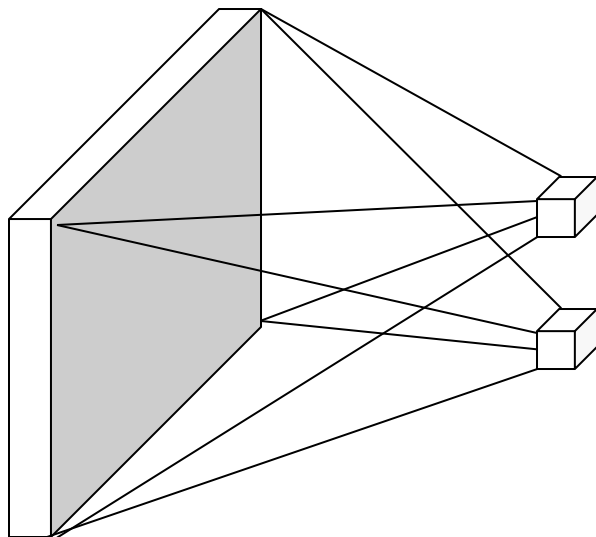
image

Fully connected layer



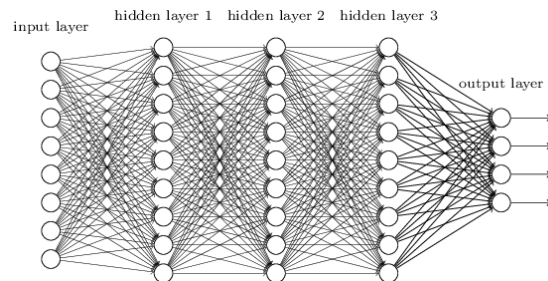
- This kind of design is known as *multi-layer perceptron* (MLP)

A Neural Network for Images



image

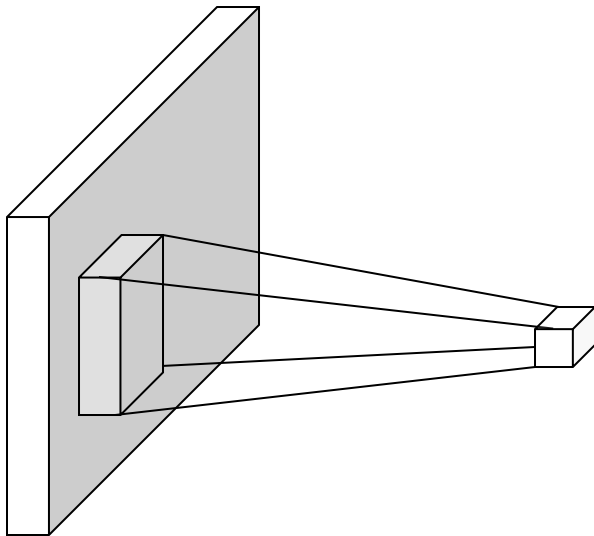
Fully connected layer



- This kind of design is known as *multi-layer perceptron* (MLP)
- What is wrong with this?

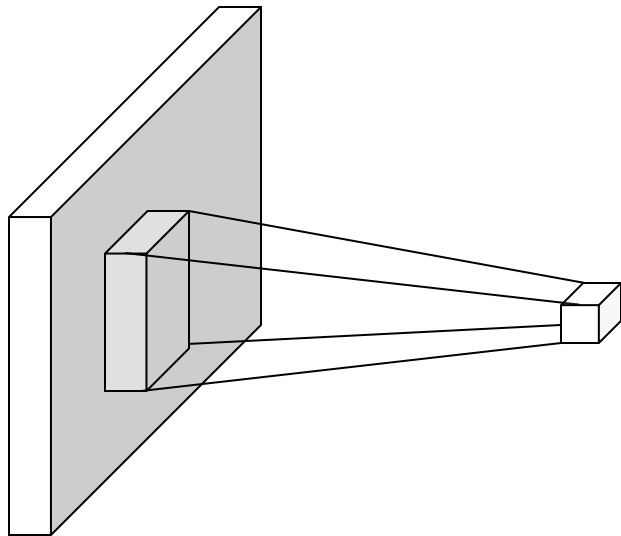


Recall: MLP as
bank of whole-image
templates



image

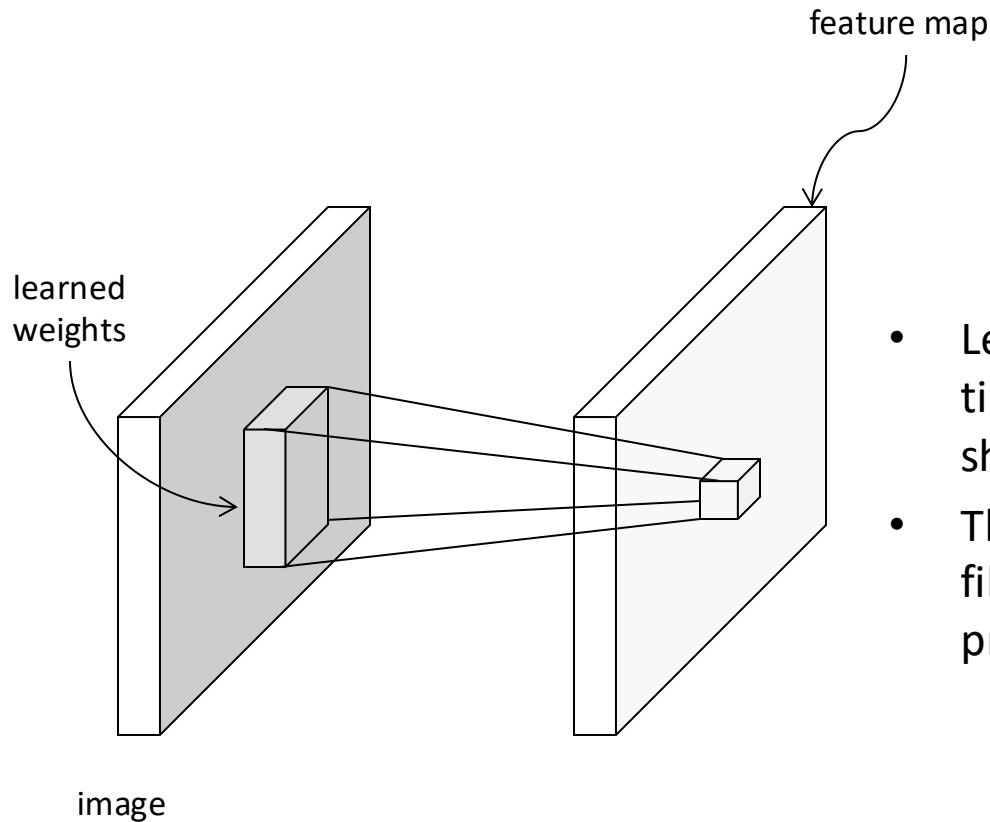
- Let's limit the *receptive fields* of units, tile them over the input image, and share their weights



image

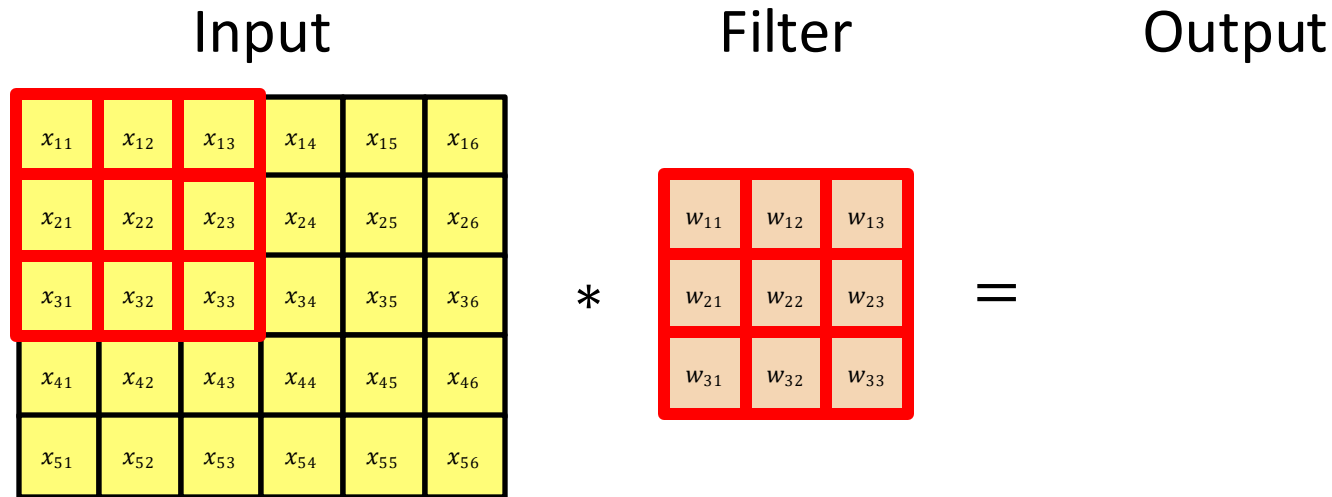
- Let's limit the *receptive fields* of units, tile them over the input image, and share their weights

Convolutional Architecture

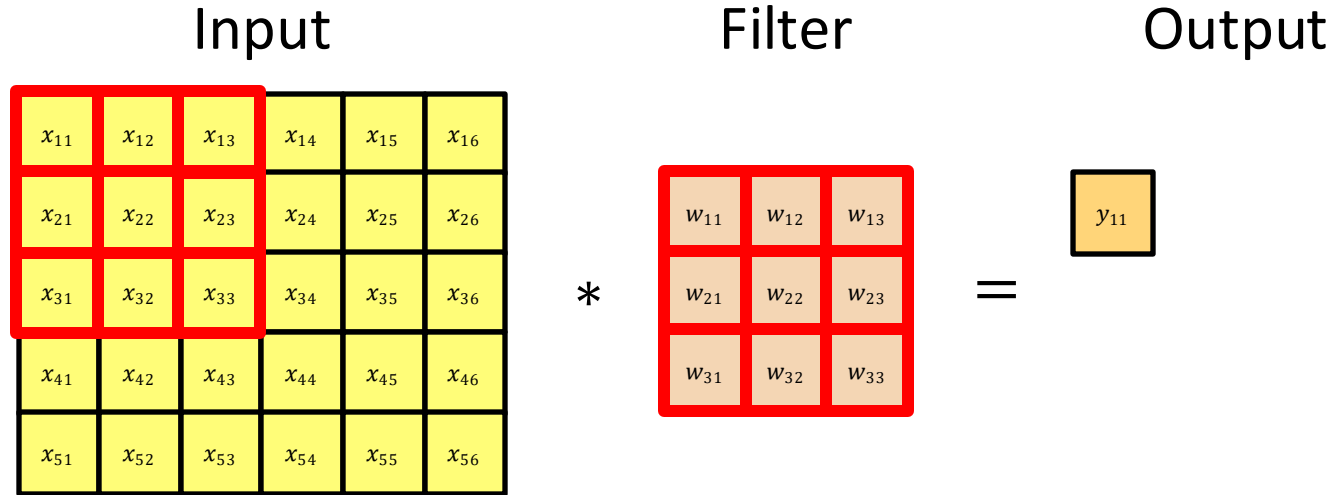


- Let's limit the *receptive fields* of units, tile them over the input image, and share their weights
- This is equivalent to sliding the learned filter over the image, computing dot products at every location

Convolutional Architecture

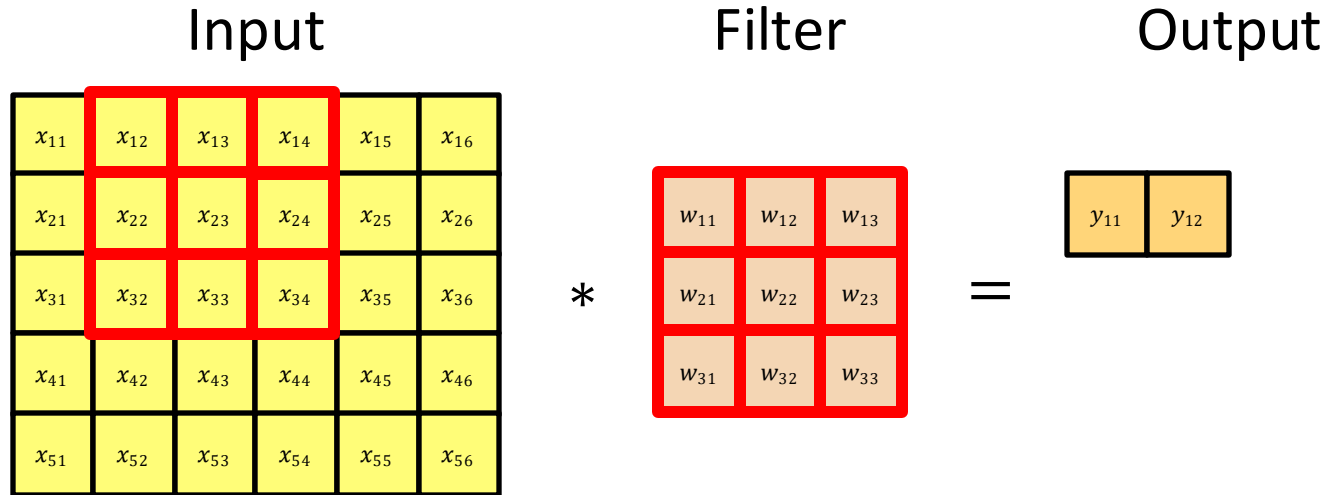


Convolutional Architecture



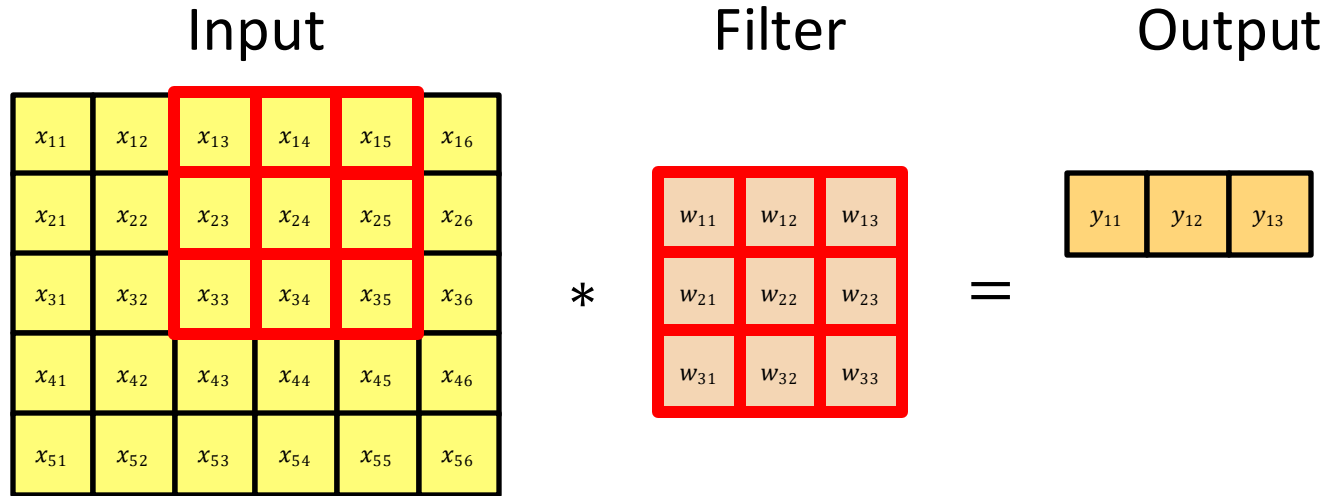
$$y_{11} = x_{11} \cdot w_{11} + x_{12} \cdot w_{12} + x_{13} \cdot w_{13} + \dots + x_{33} \cdot w_{33}$$

Convolutional Architecture



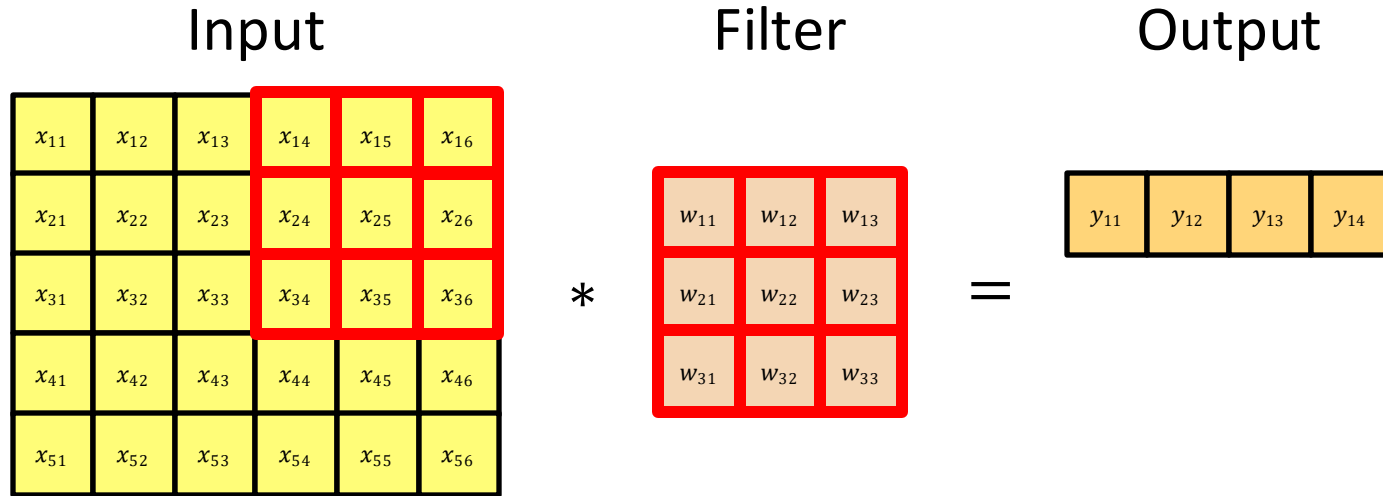
$$y_{12} = x_{12} \cdot w_{11} + x_{13} \cdot w_{12} + x_{14} \cdot w_{13} + \dots + x_{34} \cdot w_{33}$$

Convolutional Architecture



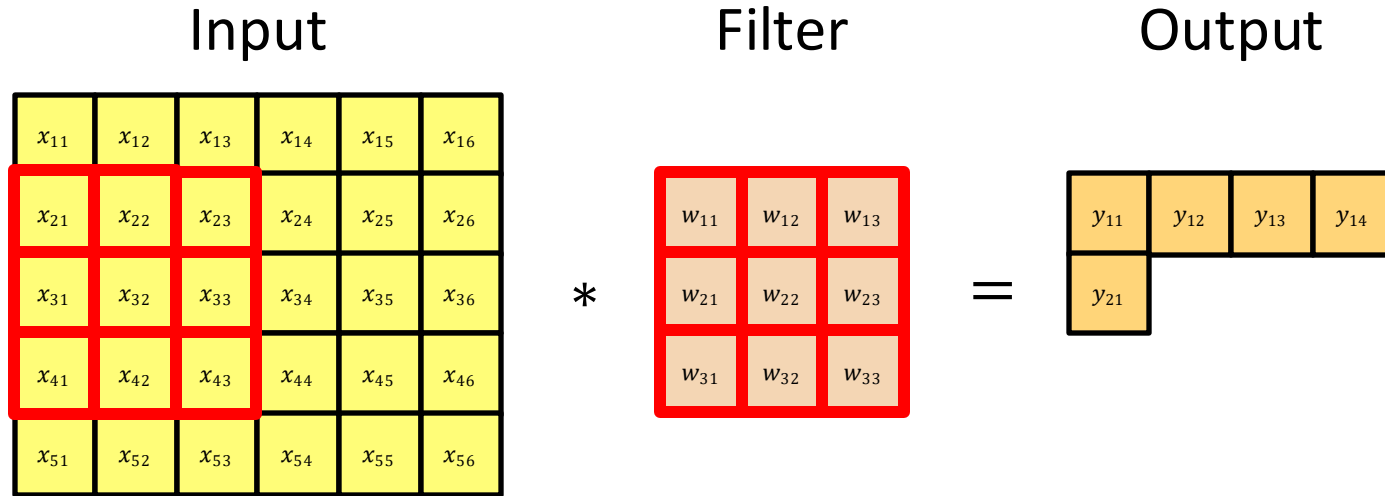
$$y_{13} = x_{13} \cdot w_{11} + x_{14} \cdot w_{12} + x_{15} \cdot w_{13} + \dots + x_{35} \cdot w_{33}$$

Convolutional Architecture



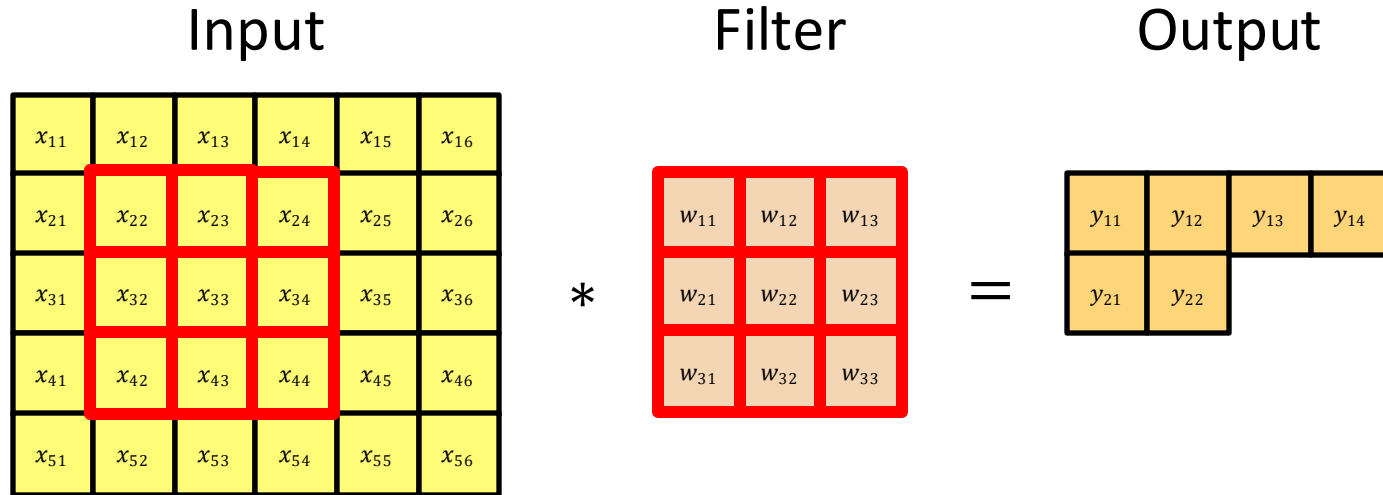
$$y_{14} = x_{14} \cdot w_{11} + x_{15} \cdot w_{12} + x_{16} \cdot w_{13} + \dots + x_{36} \cdot w_{33}$$

Convolutional Architecture



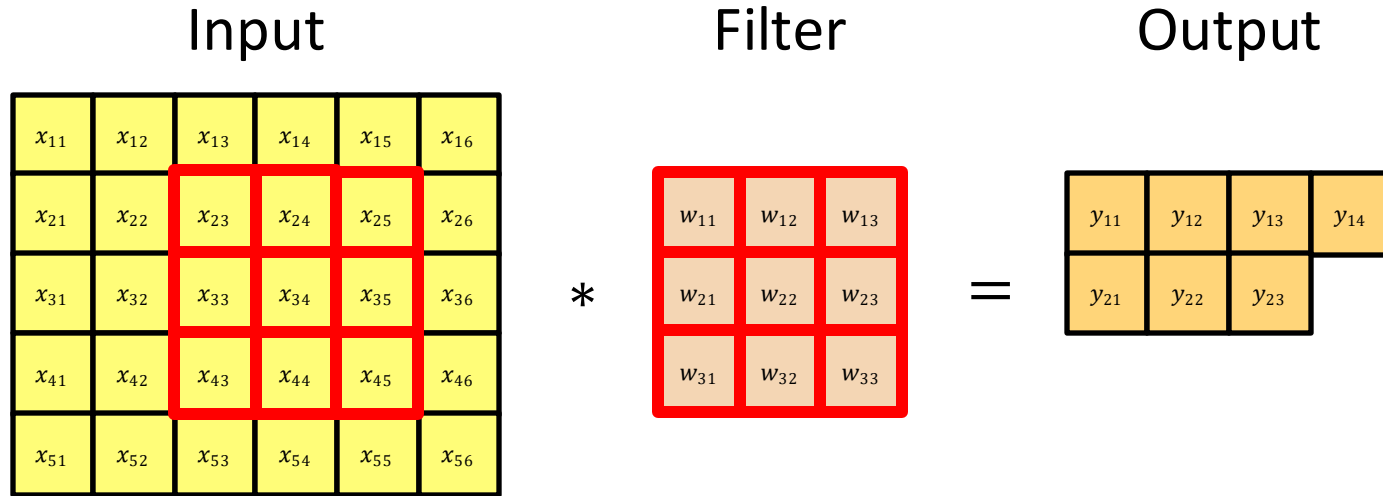
$$y_{21} = x_{21} \cdot w_{11} + x_{22} \cdot w_{12} + x_{23} \cdot w_{13} + \dots + x_{43} \cdot w_{33}$$

Convolutional Architecture



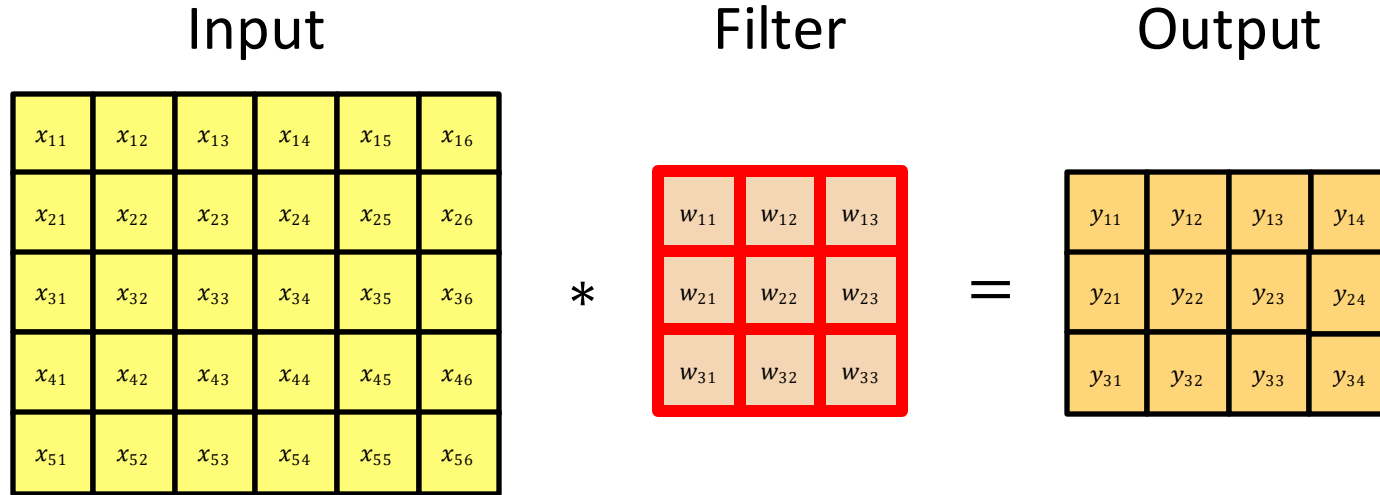
$$y_{22} = x_{22} \cdot w_{11} + x_{23} \cdot w_{12} + x_{24} \cdot w_{13} + \dots + x_{44} \cdot w_{33}$$

Convolutional Architecture

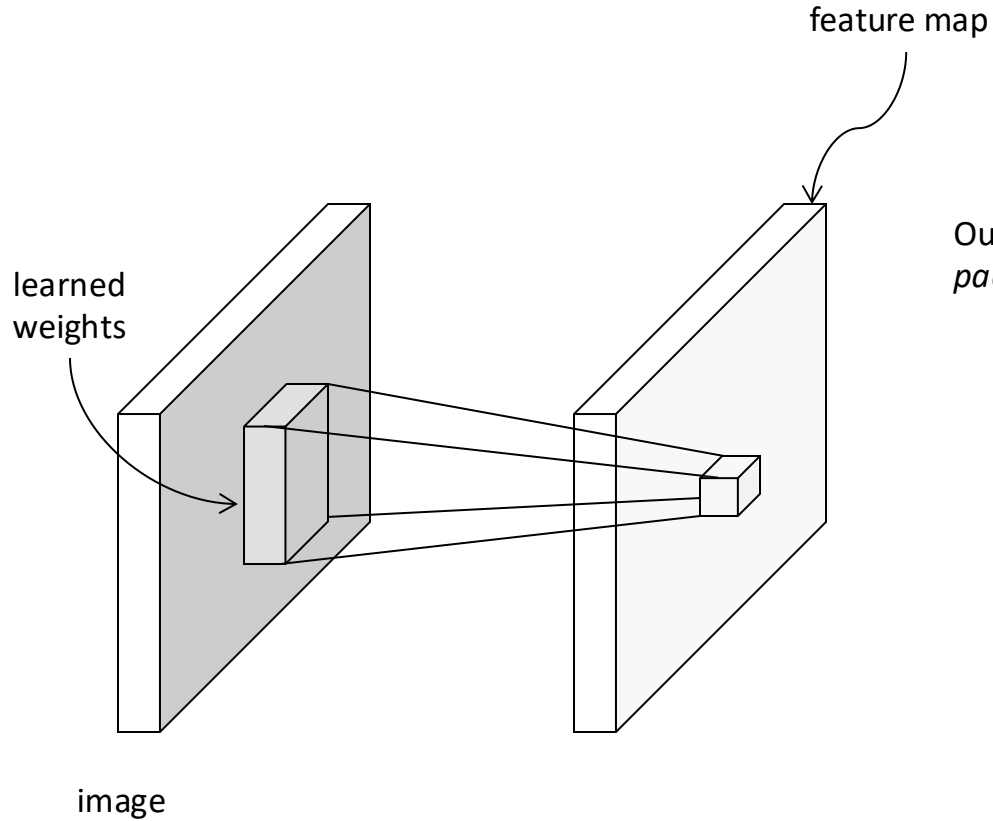


$$y_{23} = x_{23} \cdot w_{11} + x_{24} \cdot w_{12} + x_{25} \cdot w_{13} + \dots + x_{45} \cdot w_{33}$$

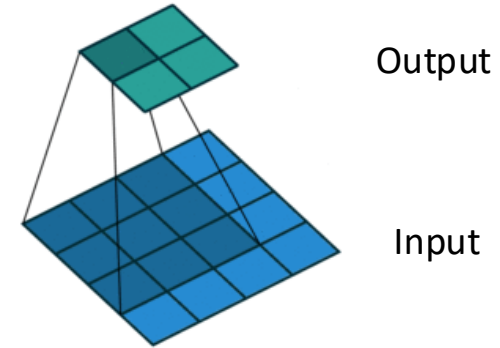
Convolutional Architecture



Convolutional Architecture

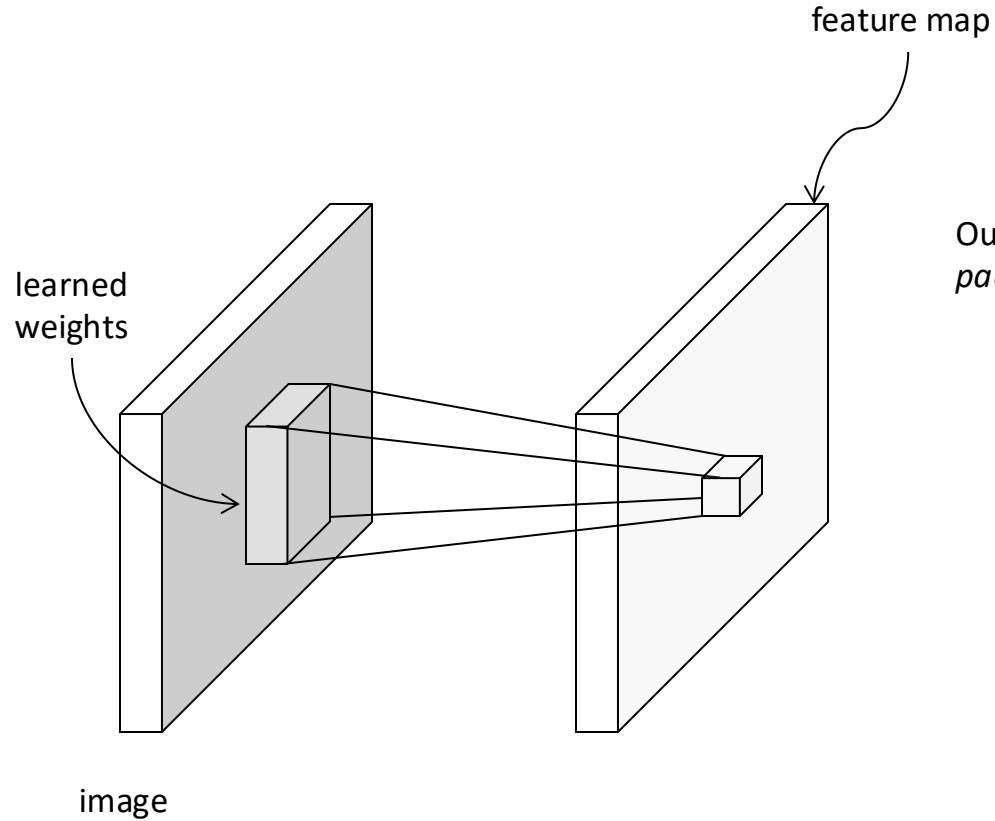


Output feature map resolution depends on *padding* and *stride*

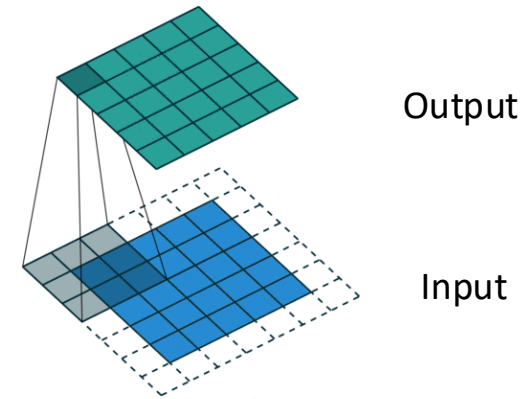


No padding, stride 1

Convolutional Architecture

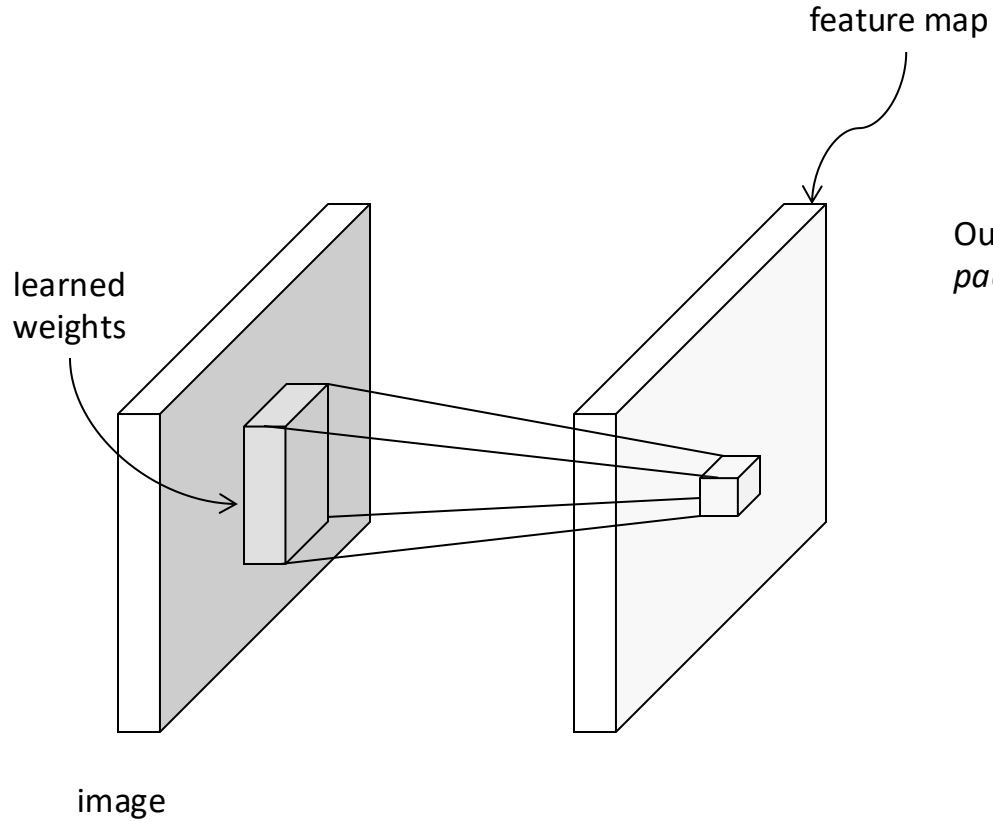


Output feature map resolution depends on *padding* and *stride*

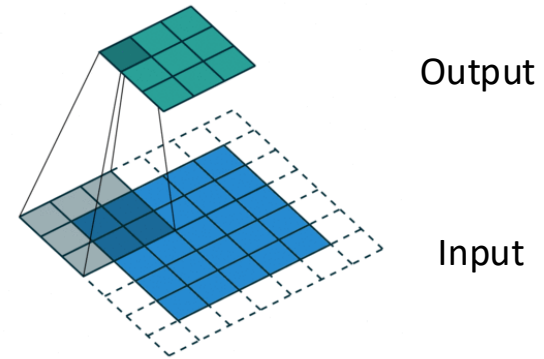


With padding, stride 1

Convolutional Architecture

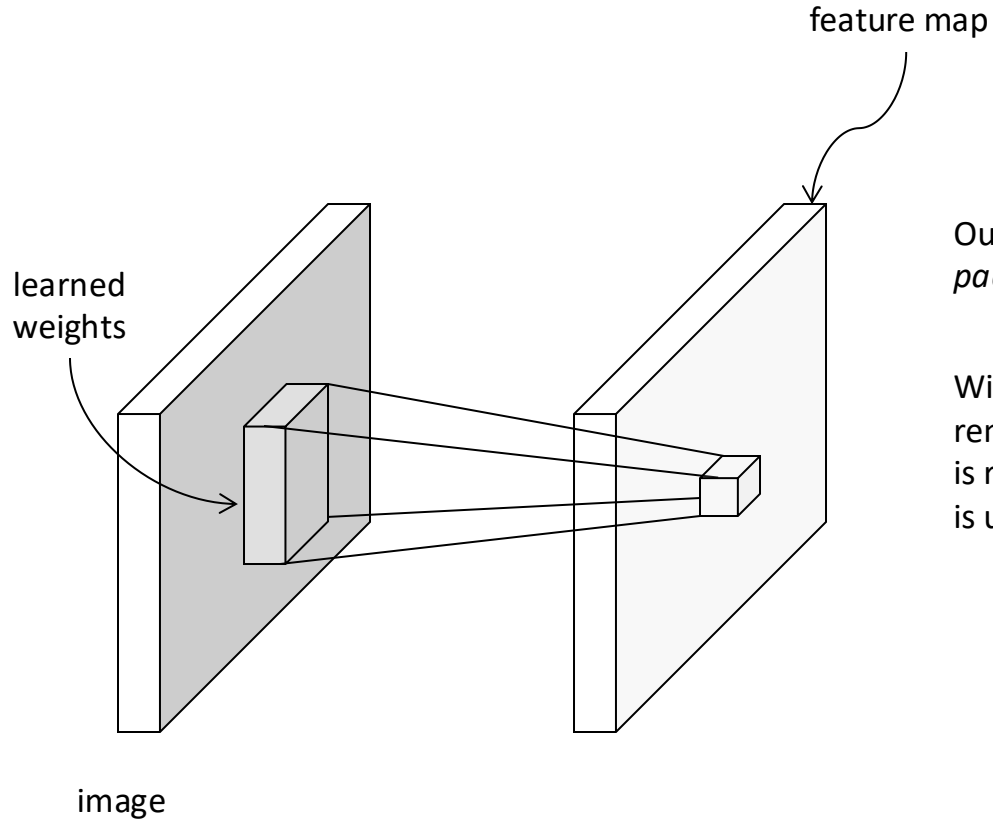


Output feature map resolution depends on *padding* and *stride*



With padding, stride 2

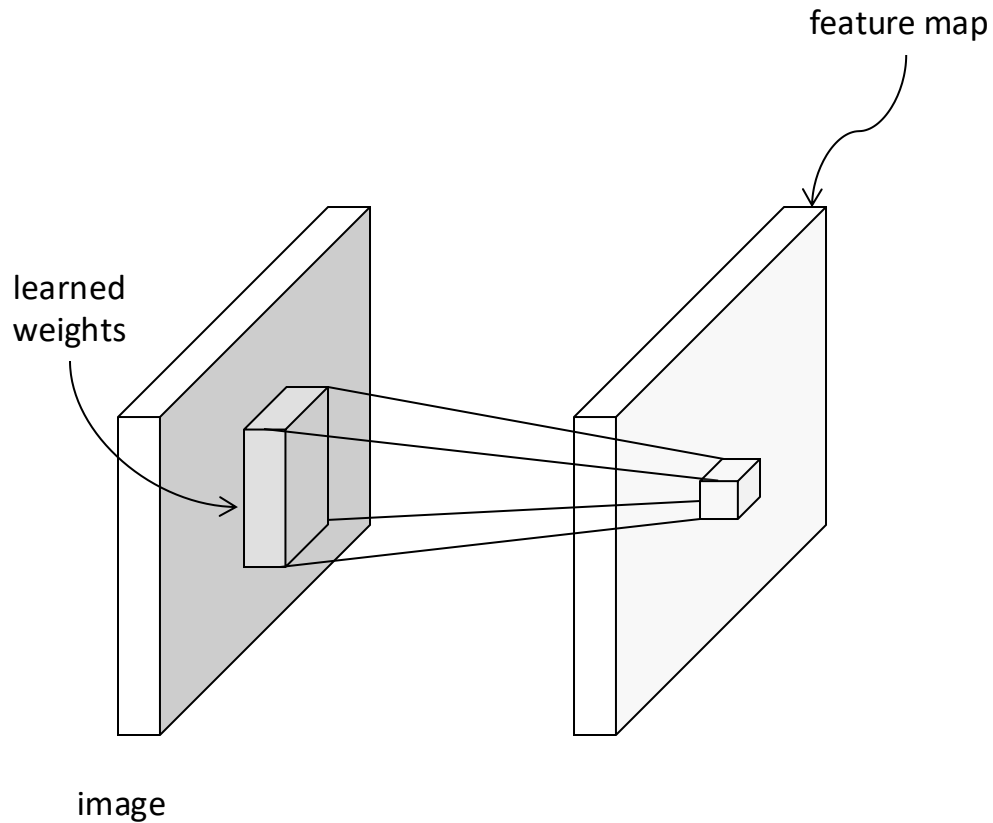
Convolutional Architecture



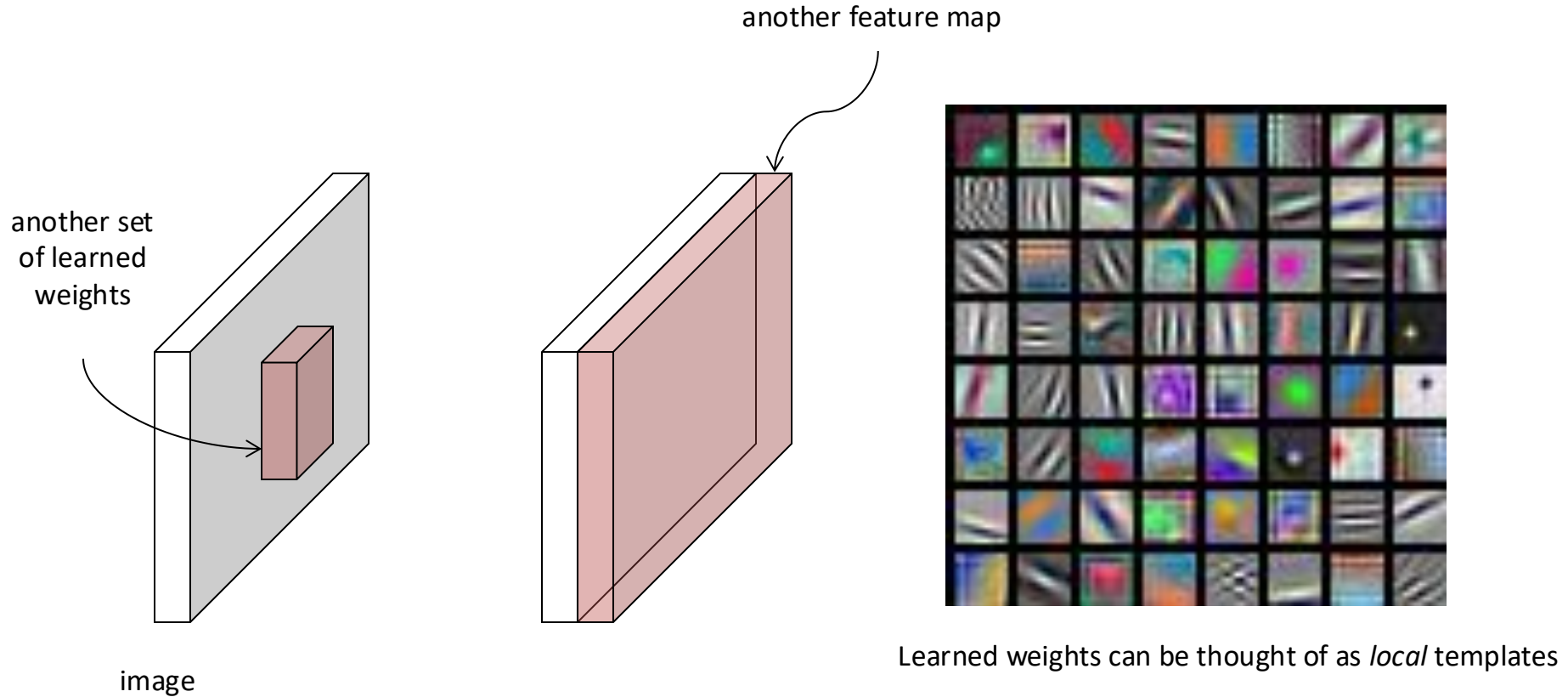
Output feature map resolution depends on *padding* and *stride*

With padding, spatial resolution remains the same if stride of 1 is used, is reduced by factor of $1/S$ if stride of S is used

Convolutional Architecture

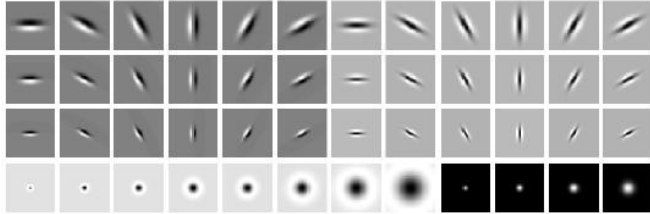


Convolutional Architecture

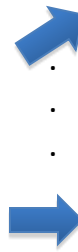


Convolution and Traditional Feature Extraction

bank of K filters



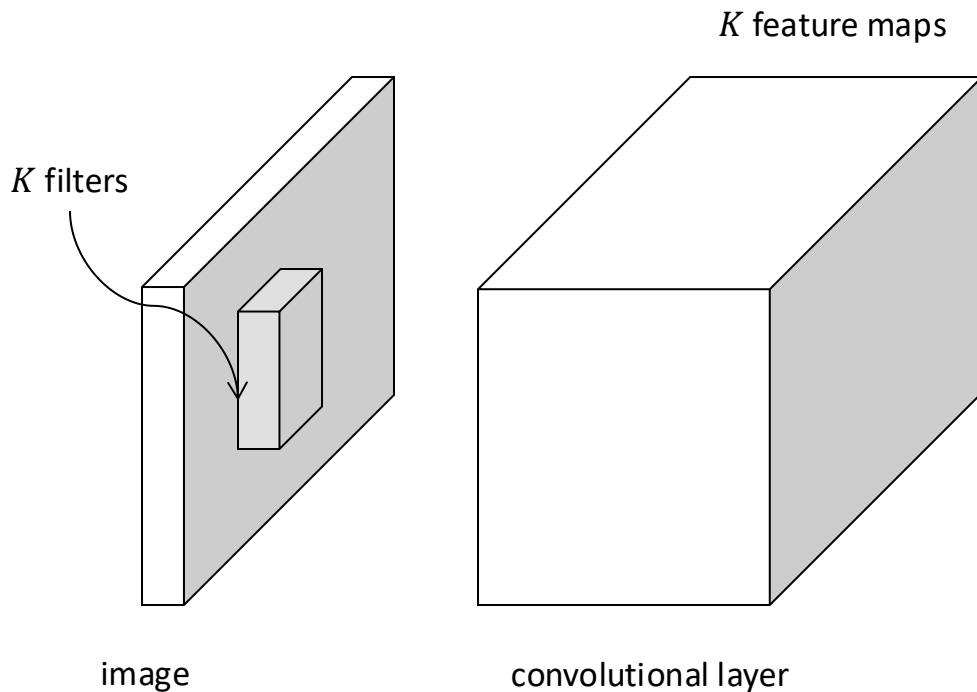
K feature maps



image

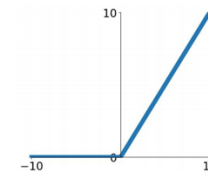
feature map

Elementwise Nonlinearity



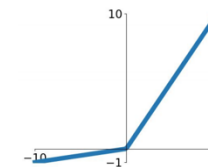
Almost always directly followed by a Rectified Linear Unit (ReLU):

$$\max(0, x)$$



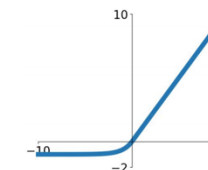
Some alternatives to ReLU:

Leaky ReLU
 $\max(0.1x, x)$

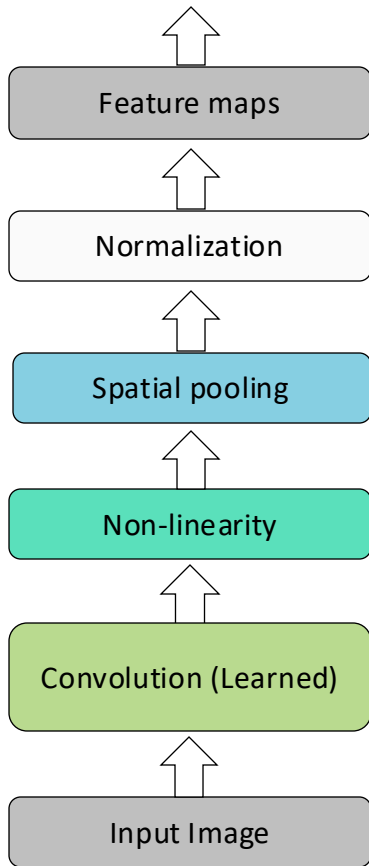


ELU

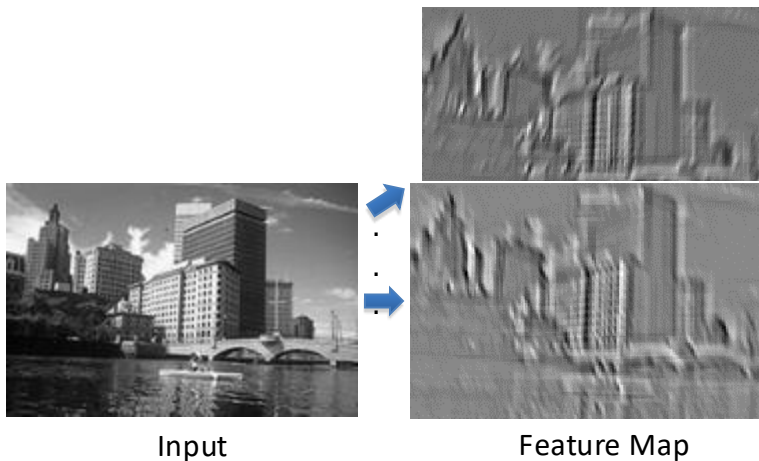
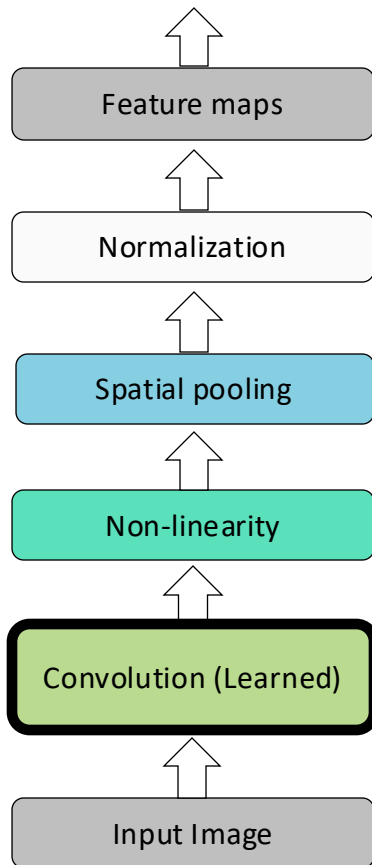
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



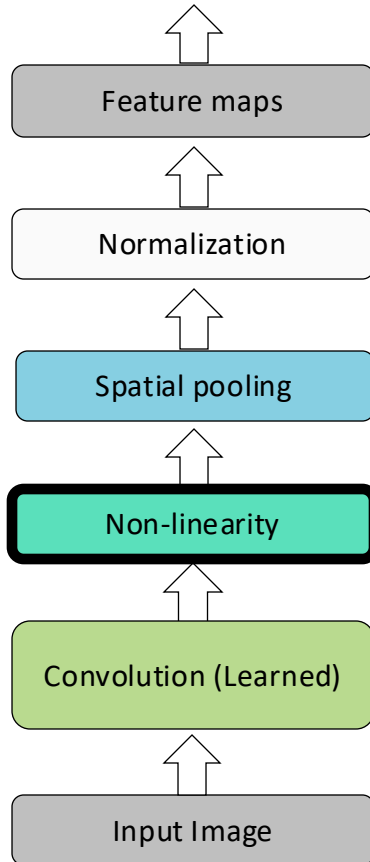
Convolutional Neural Networks



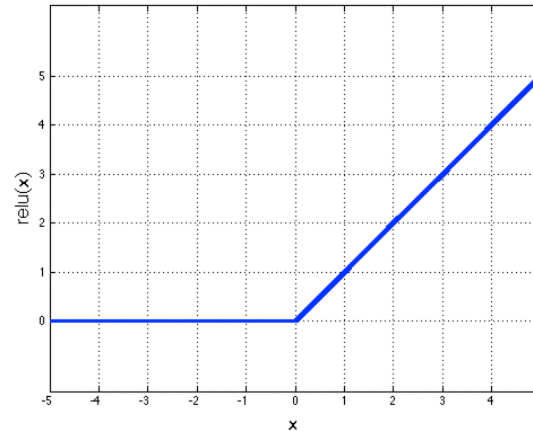
Convolutional Neural Networks



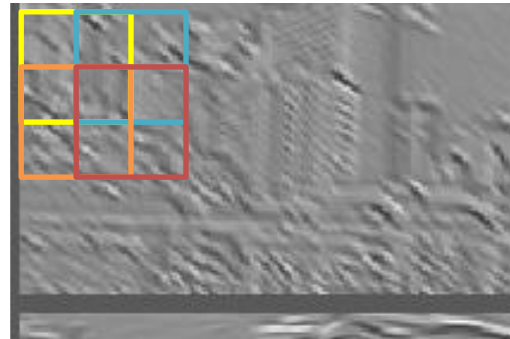
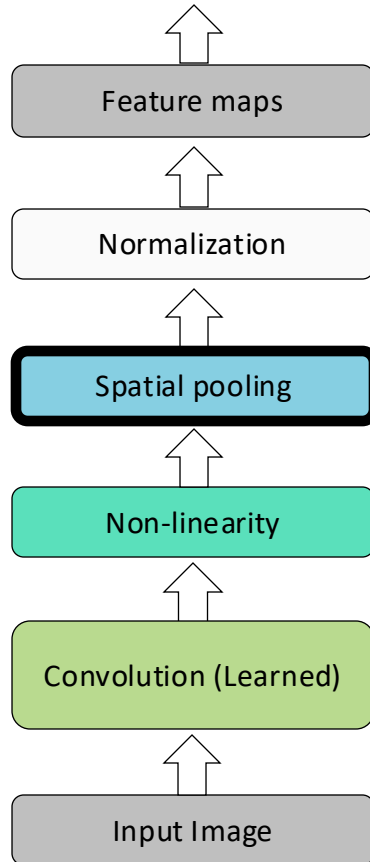
Convolutional Neural Networks



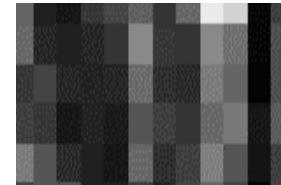
Rectified Linear Unit (ReLU)



Convolutional Neural Networks



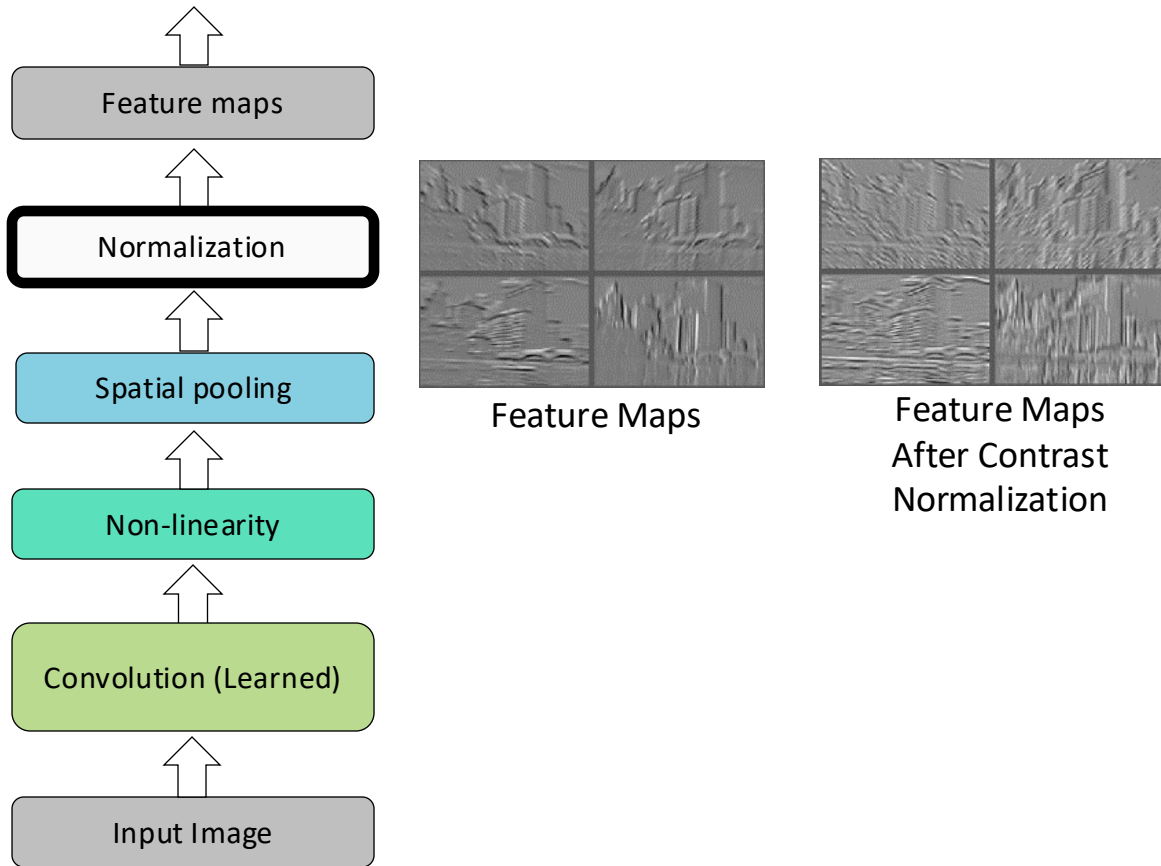
Max pooling



Max-pooling: a non-linear down-sampling

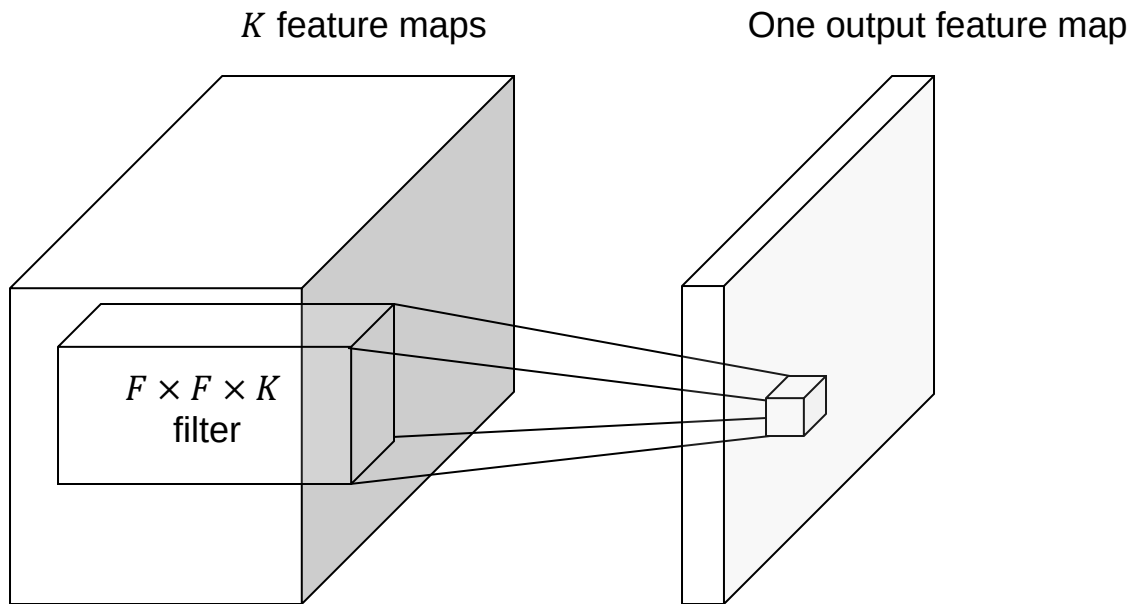
Provide *local invariance*

Convolutional Neural Networks



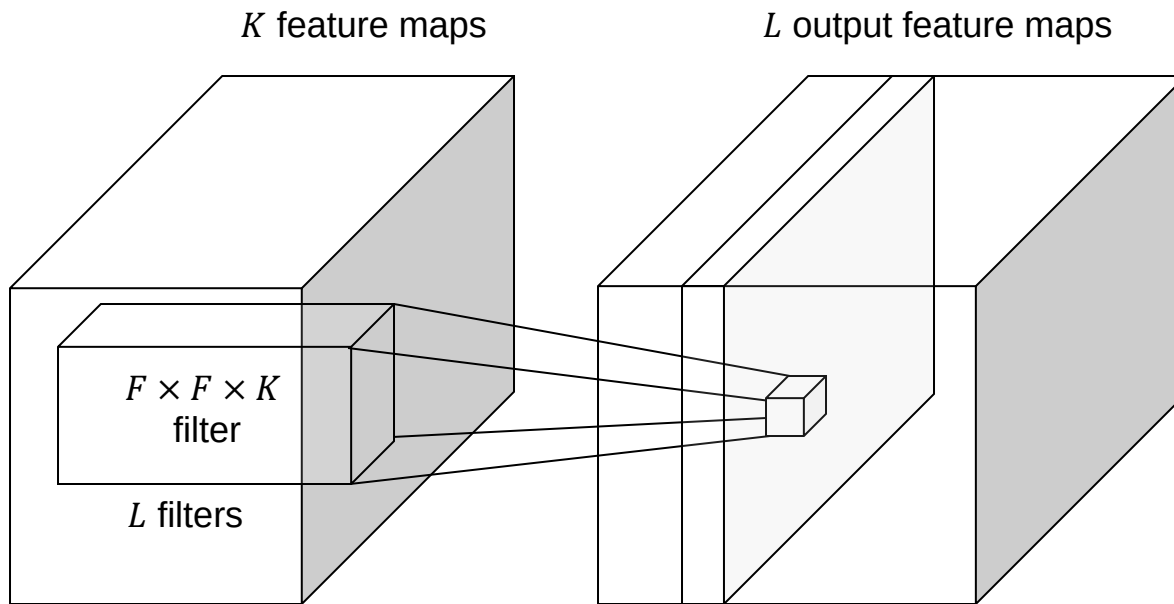
Three-dimensional Convolutions

- What if the *input* to a convolutional layer is a stack of K feature maps?

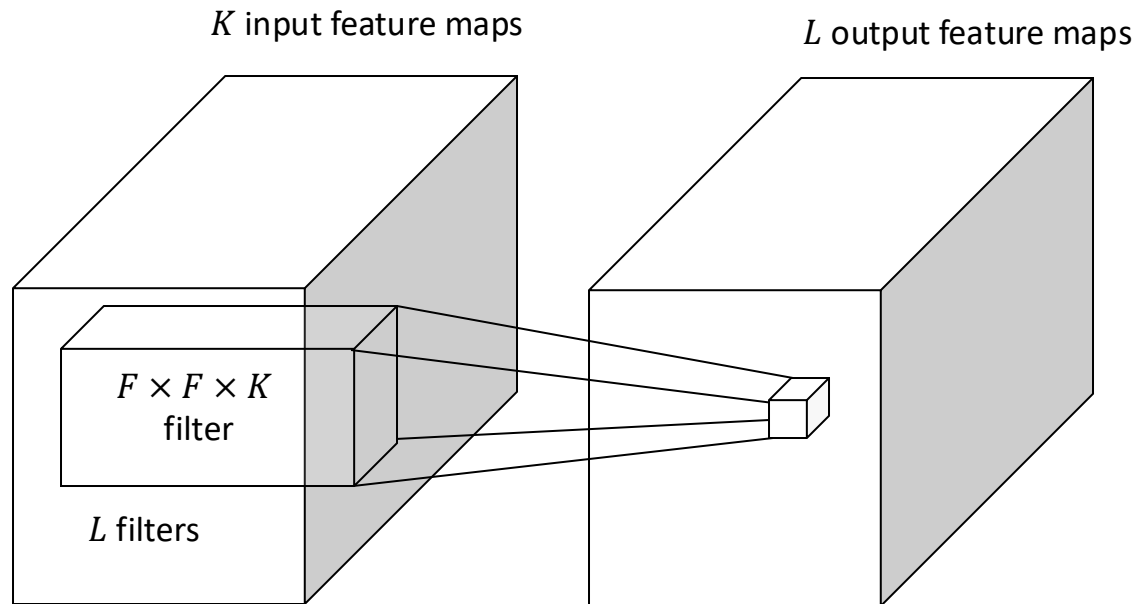


Three-dimensional Convolutions

- What if the *input* to a convolutional layer is a stack of K feature maps?

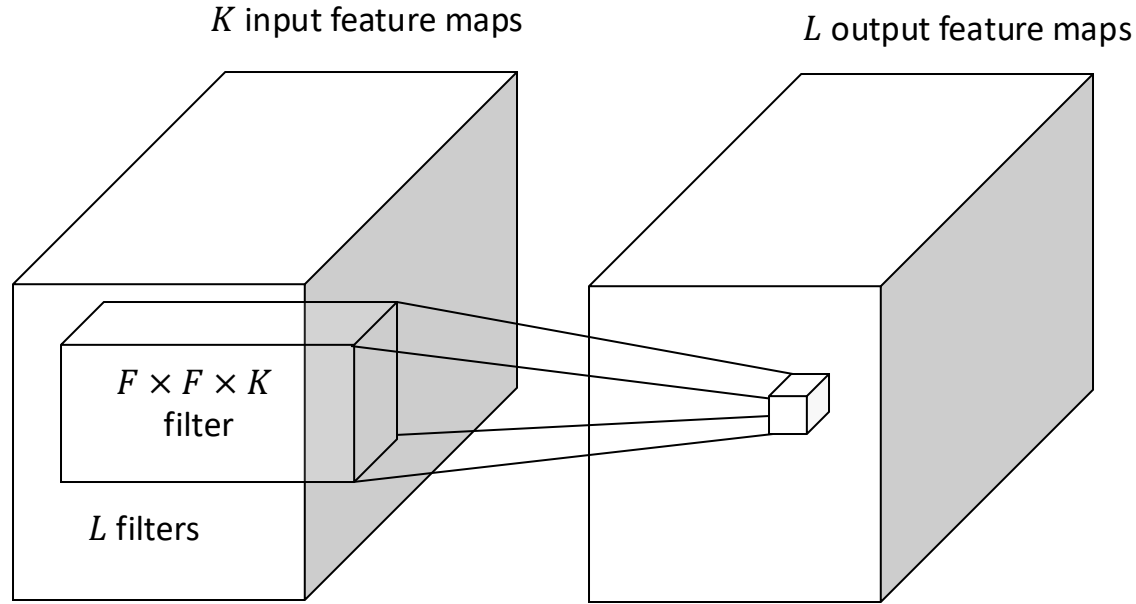


Convolutional Layer: Computational Cost



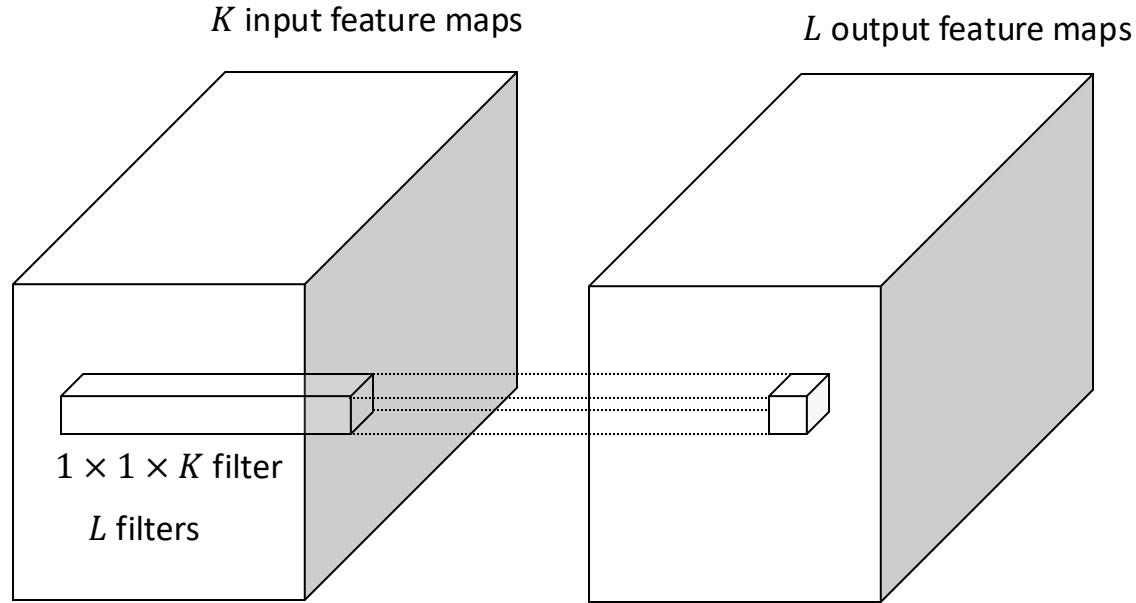
- Assuming the input feature maps have spatial resolution $H \times W$, how many operations are needed to compute the output feature volume?
 - $F^2 K L H W$

1x1 Convolutional Layer



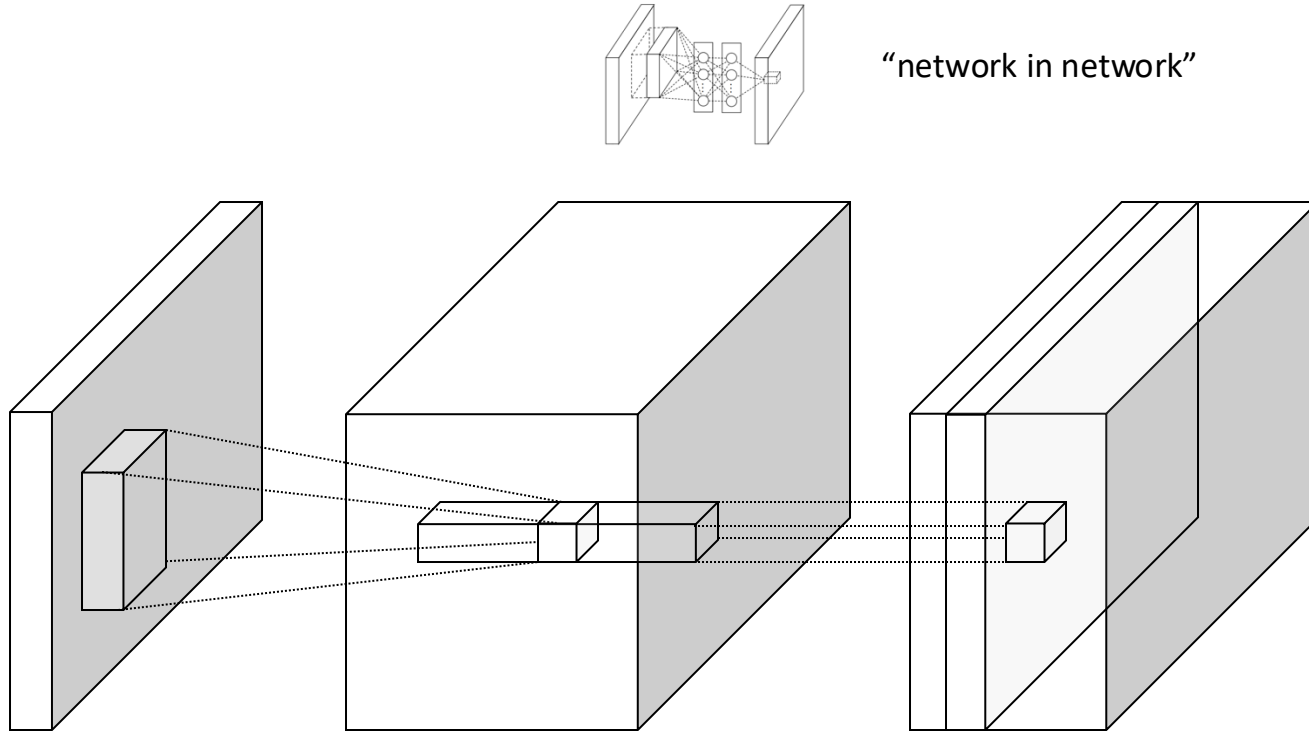
What if we make $F = 1$?

1x1 Convolutional Layer



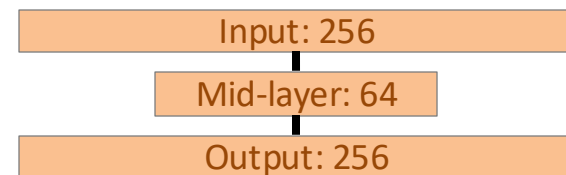
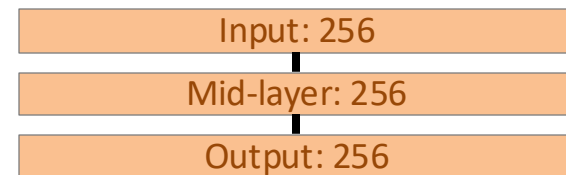
What if we make $F = 1$?

1x1 Convolutional Layer

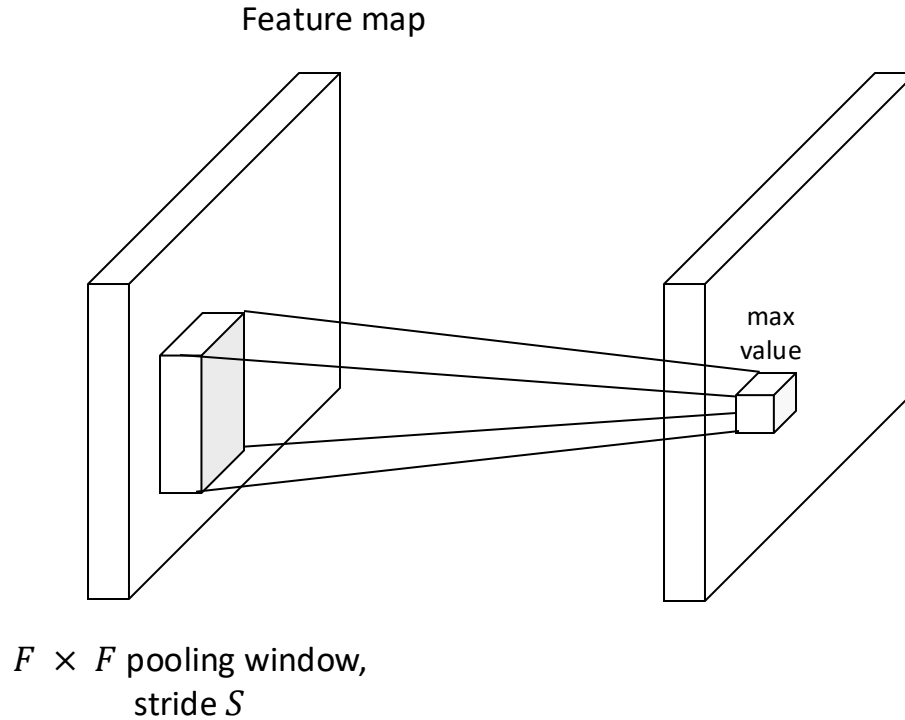


Why 1x1 convolutions?

- **Option 1:** 3×3 conv layer with 256 channels at input and output
 - $256 \times 256 \times 3 \times 3$
 - Total $\approx 600,000$ weights & operations (per location)
- **Option 2: “bottleneck module”**
 - 1×1 conv layer, $256 \rightarrow 64$ channels
 - 3×3 conv layer, $64 \rightarrow 64$ channels
 - 1×1 conv layer, $64 \rightarrow 256$ channels
 - $256 \times 64 \times 1 \times 1 \approx 16,000$
 - $64 \times 64 \times 3 \times 3 \approx 36,000$
 - $64 \times 256 \times 1 \times 1 \approx 16,000$
 - Total $\approx 70,000$ weights & operations



Max Pooling Layer



Usually: $F = 2$ or 3 , $S = 2$

Max Pooling: Example

Single channel

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

Max pooling with 2×2
kernel size and stride 2



Max Pooling: Example

Single channel

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

Max pooling with 2×2
kernel size and stride 2



Max Pooling: Example

Single channel

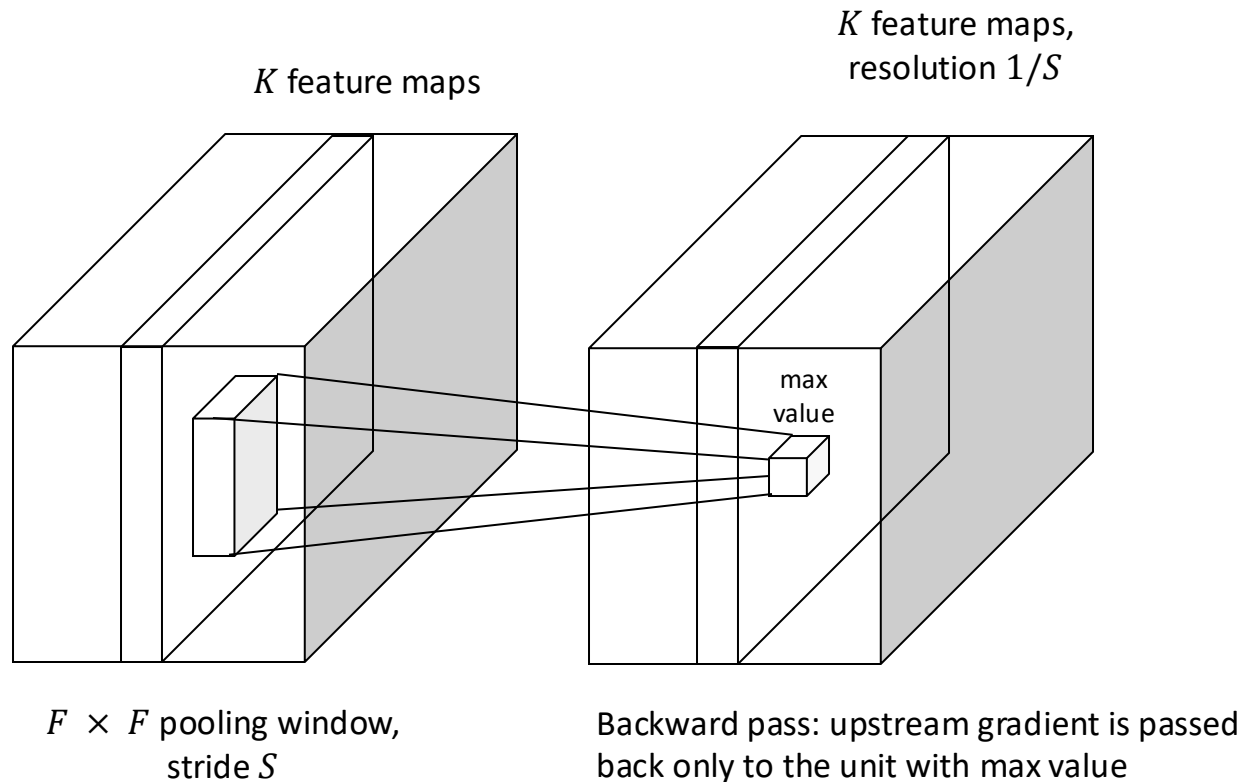
1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

Max pooling with 2×2
kernel size and stride 2



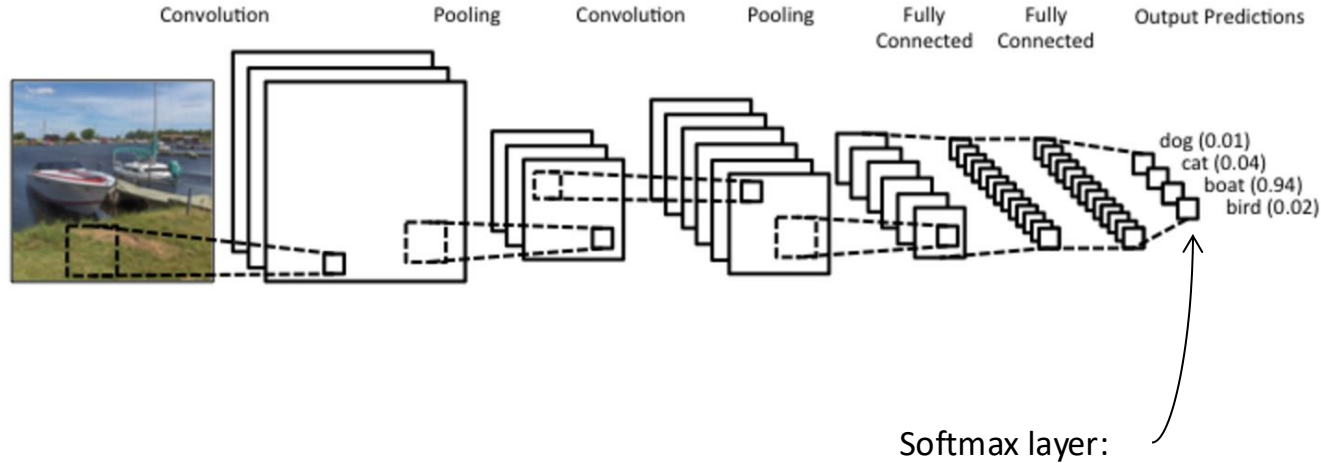
6	8
3	4

Convolutional Neural Networks

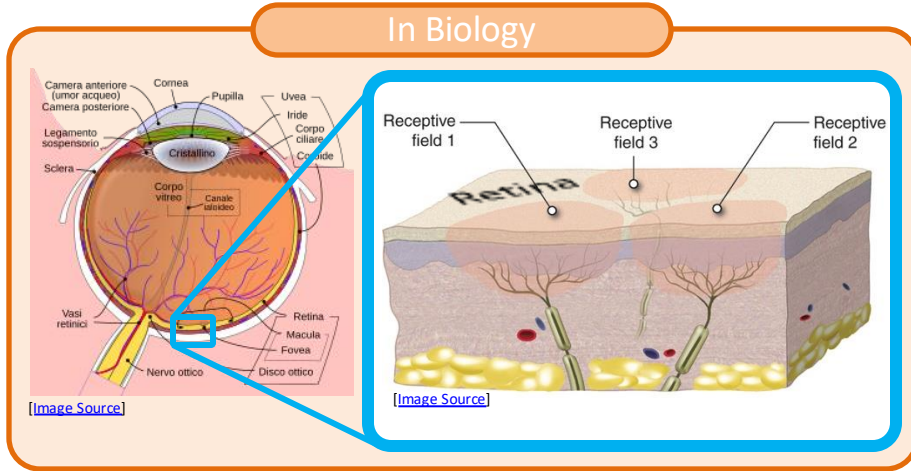


Usually: $F = 2$ or 3 , $S = 2$

Simplified CNN Pipeline



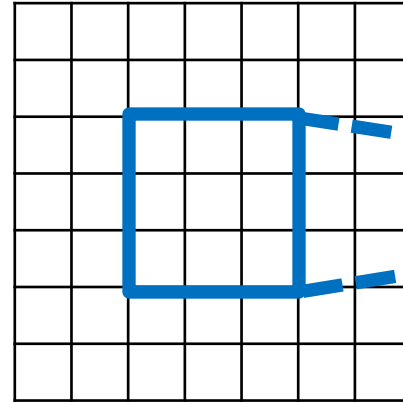
Receptive Field



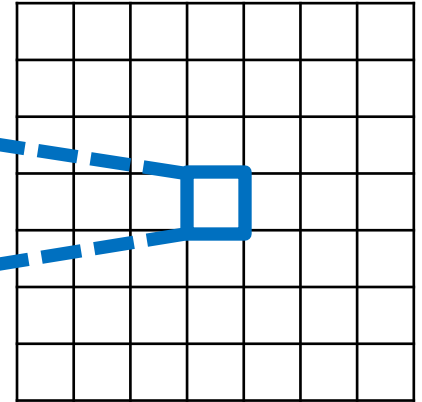
The *receptive field* of a unit is the region of the input feature map whose values contribute to the response of that unit (either in the previous layer or in the initial image)

3x3 convolutions, stride 1

Input



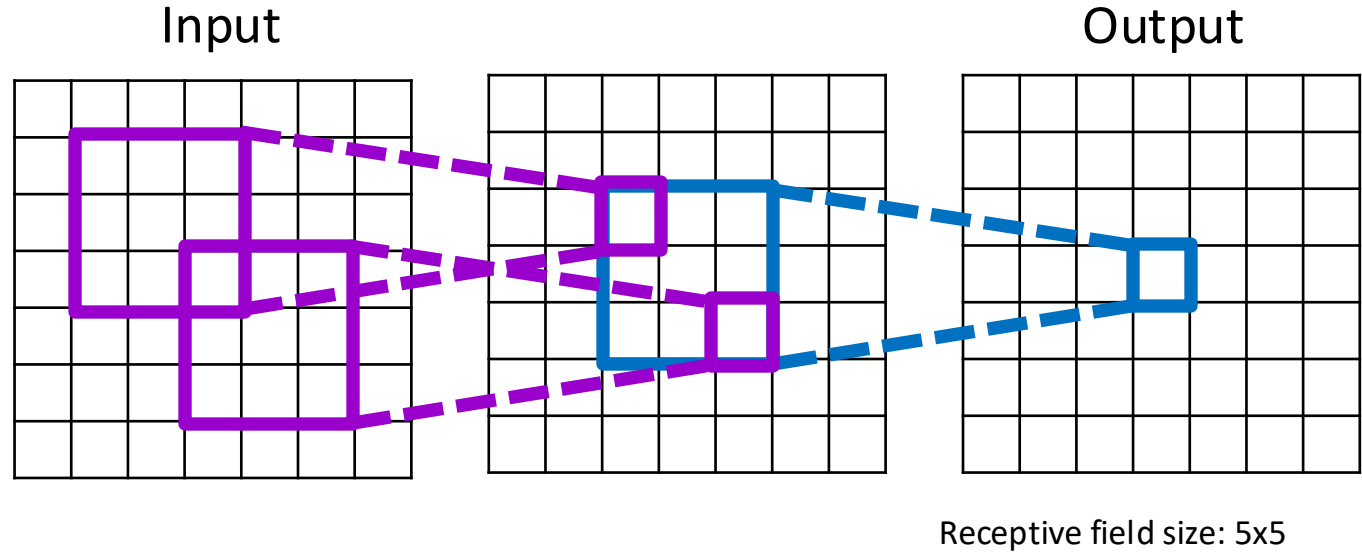
Output



Receptive field size: 3x3

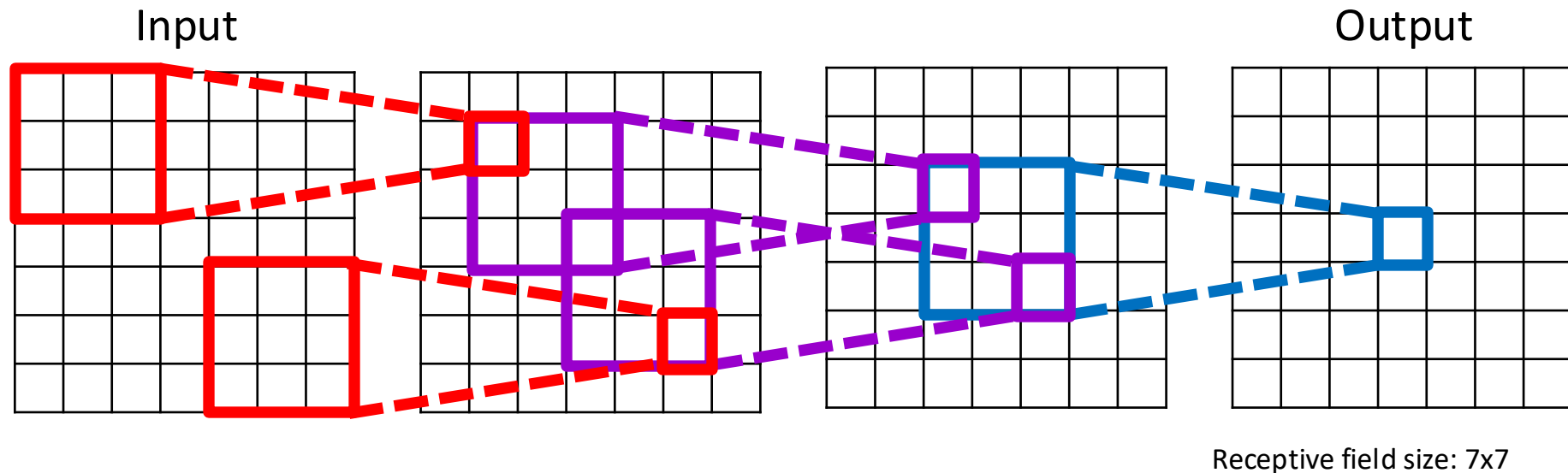
Receptive Field

3x3 convolutions, stride 1



Receptive Field

3x3 convolutions, stride 1

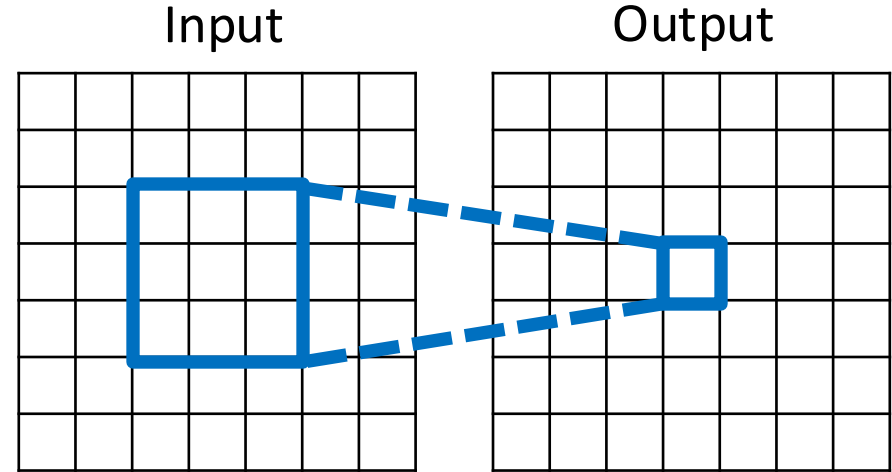


Each successive convolution adds $F - 1$ to the receptive field size

With L layers the receptive field size is $1 + L * (F - 1)$

Receptive Field

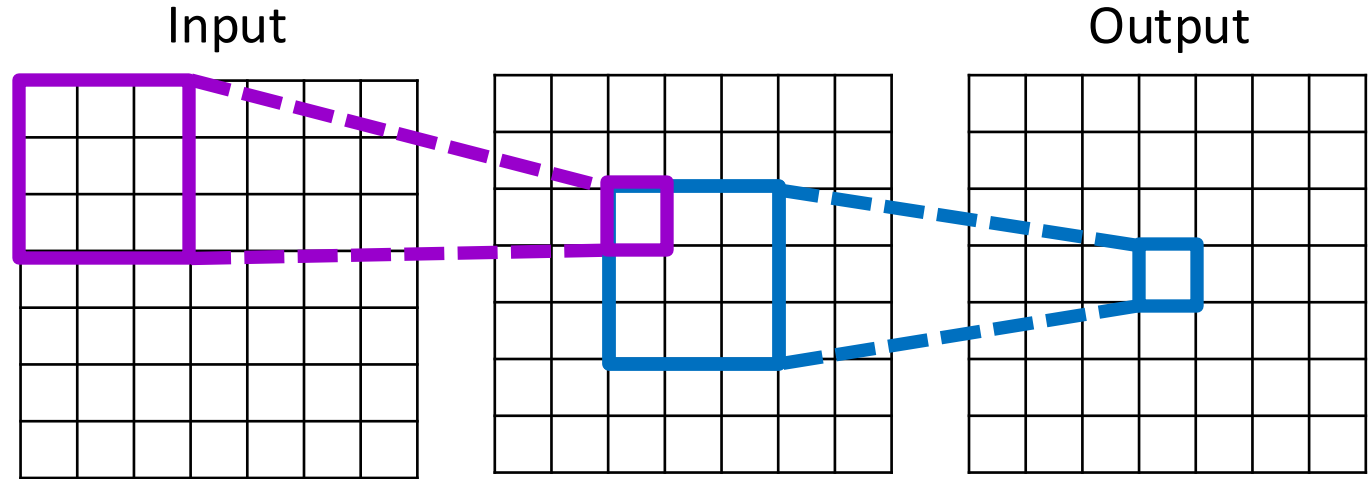
3x3 convolutions, stride 2



Receptive field size: 3x3

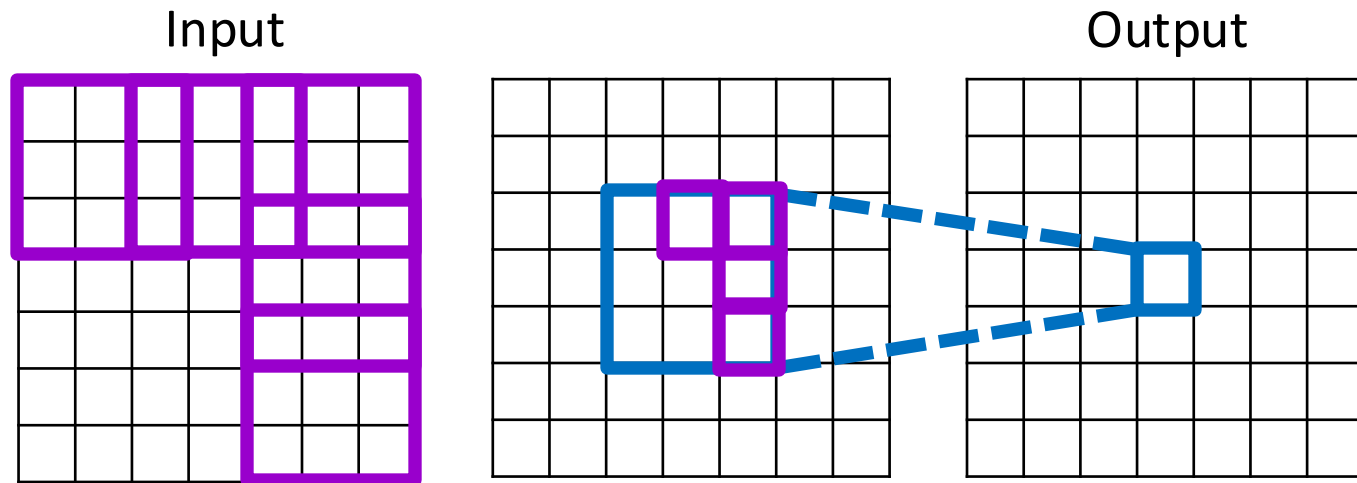
Receptive Field

3x3 convolutions, stride 2



Receptive Field

3x3 convolutions, stride 2

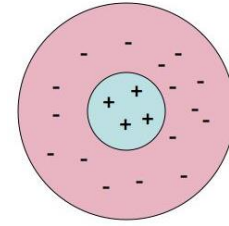


Receptive field size: 7x7

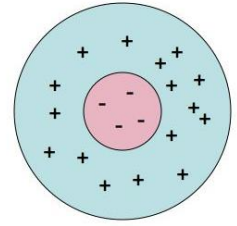
With a stride of 2, receptive field size is given by $2^{L+1} - 1$, i.e., it grows exponentially (though spatial resolution decreases exponentially)

Receptive Fields in Biology

- Biological inspiration: D. Hubel and T. Wiesel (1959, 1962, Nobel Prize 1981)
 - Visual cortex consists of a hierarchy of *simple*, *complex*, and *hyper-complex* cells

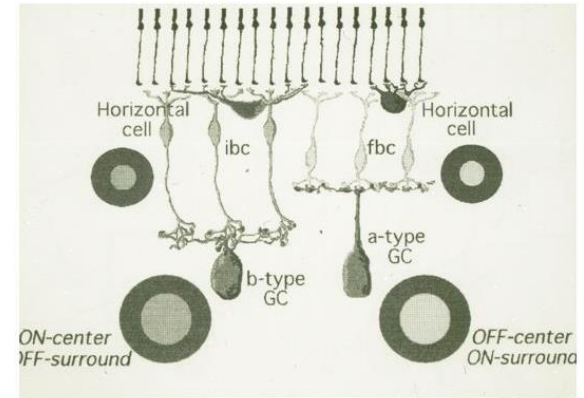


On-center, Off-surround

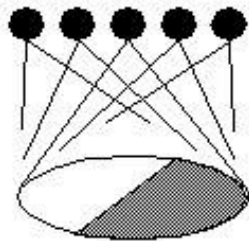


Off-center, On-surround

On-off layer segregation



Hubel & Wiesel topographical mapping

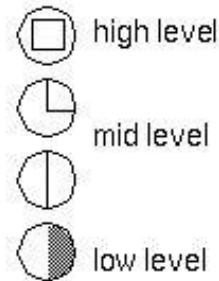
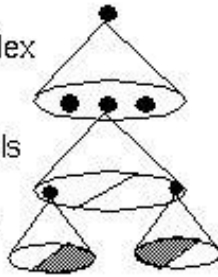


featural hierarchy

hyper-complex cells

complex cells

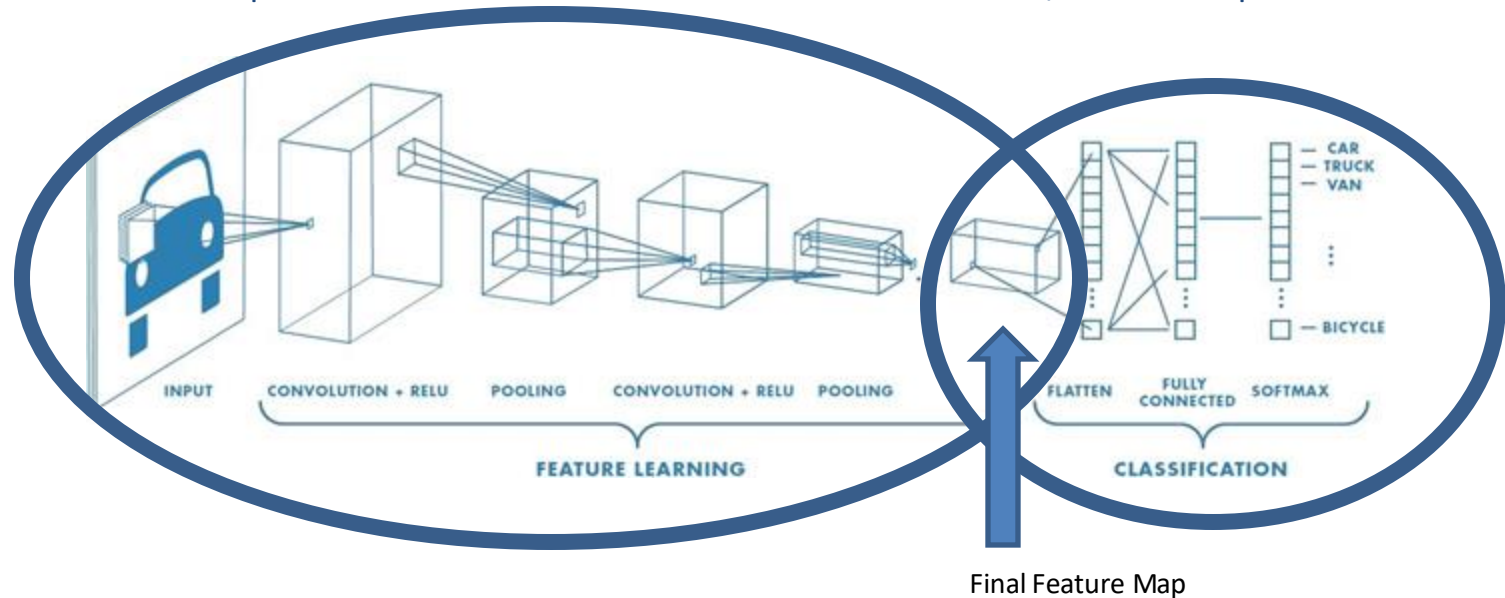
simple cells



Typical ConvNet Architecture

ConvNet architecture consists of sequence of convolutional blocks (convolution + relu + pooling + normalisation):

- First block convolves image into activations (depth = nr of filters)
- Consequent blocks convolve activations to new activations / feature maps

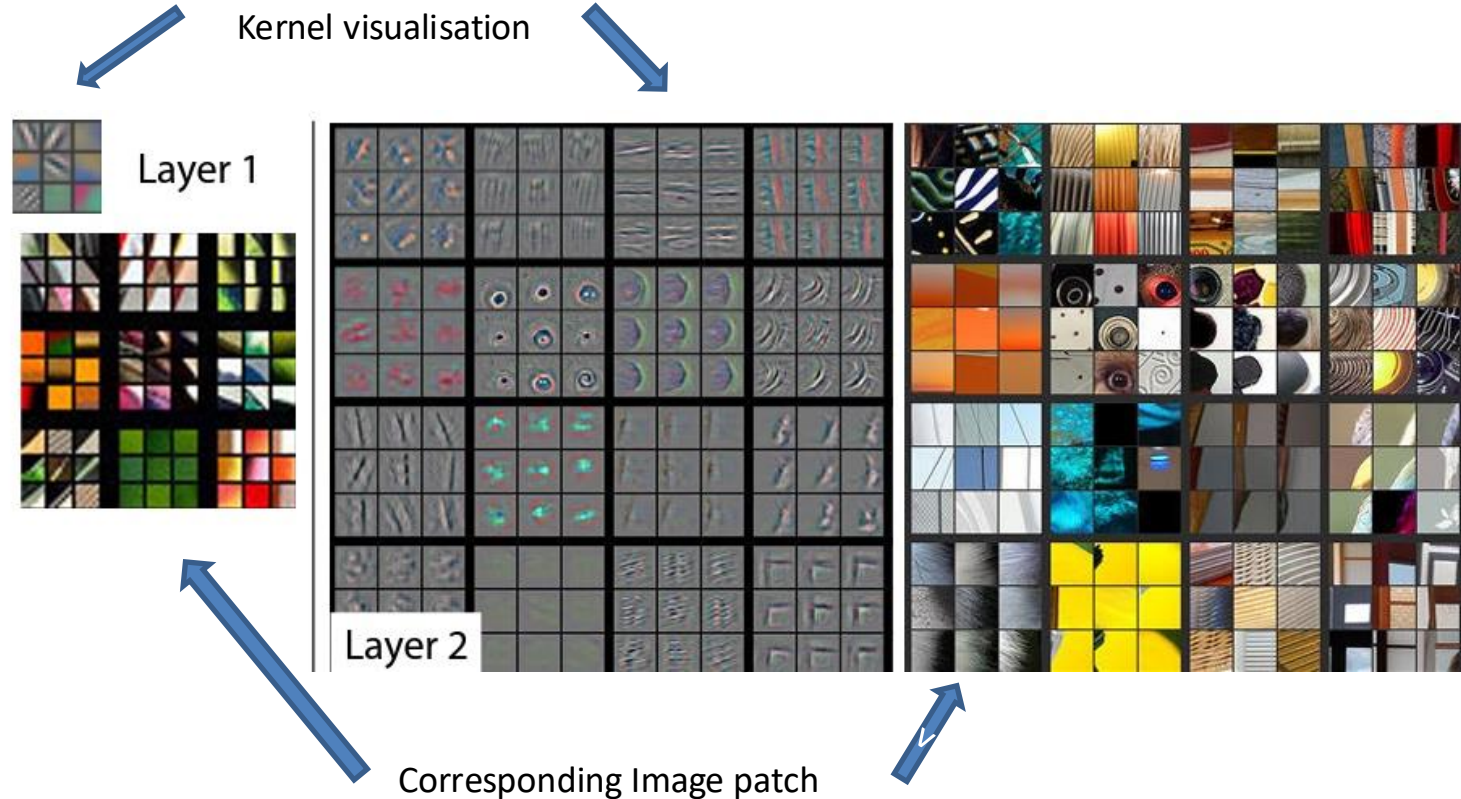


The final feature map is input for Classification

- Typical with fully connected layers

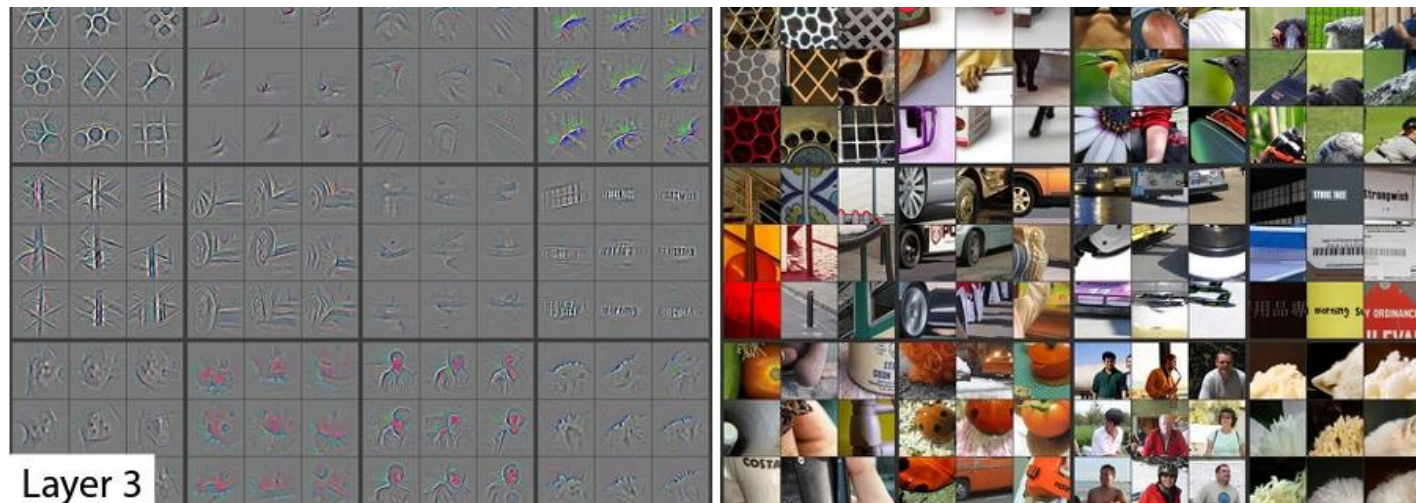
ConvNet Features

First layers learn basic features, later layers learn complex features



ConvNet Features

First layers learn basic features, later layers learn complex features

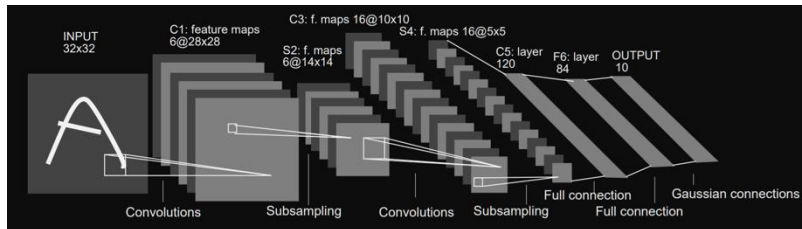


ConvNet Architectures

LeNet (Le Cun et al 1989, Bell lab) was first well known ConvNet.

Used back propagation.

Used for recognition of handwritten zipcode / checks (MNIST dataset)



ConvNet Architecture with back propagation

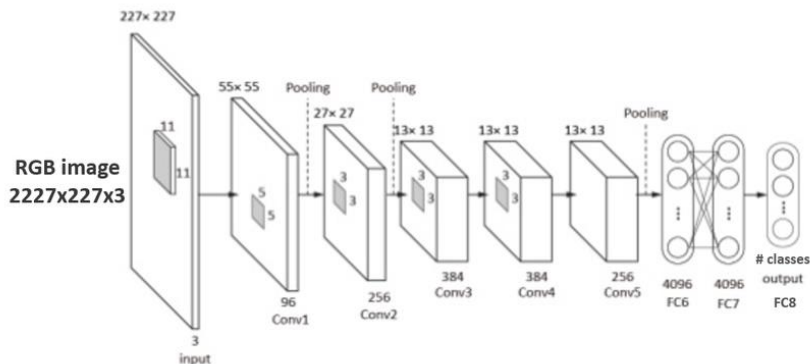
- 2 convolutional layers
- reduces spacial dimensions: 28x28x1 (black & white image) $\rightarrow \dots \rightarrow 5 \times 5$
- increases depth: 1 (black and white) $\rightarrow \dots \rightarrow 16$ (nr of filters)

Issue: slow learning speed (CPU)

ConvNet Architectures

AlexNet (Alex Krishevsky, 2012)

2012 ImageNet Large Scale Visual Recognition Challenge: 15% top 5 error



ConvNet Architecture:

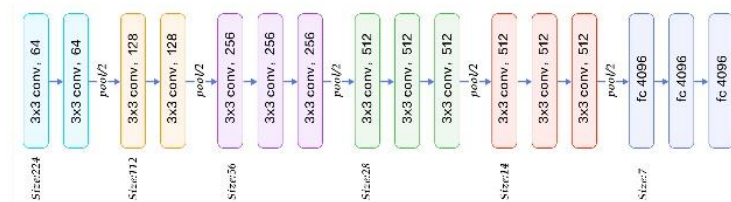
- 5 convolutional layers and 3 fully connected layers (1000 classes)
- reduces spacial dimensions: 227x227 (image) $\rightarrow \dots \rightarrow 13 \times 13$
- increases depth: 3 (image RGB depth) $\rightarrow \dots \rightarrow 256$ (nr of filters)
- use of Graphical Processing Unit (GPU)

ConvNet Architectures

VGGNet (Zimmerman, Simonyan, 2014)

The VGGNet achieved almost 92.7% top-5 test accuracy in 2014 ImageNet

VGGNet (2014)



VGGNet 16

CNN Models

Dr Mohamed Loey

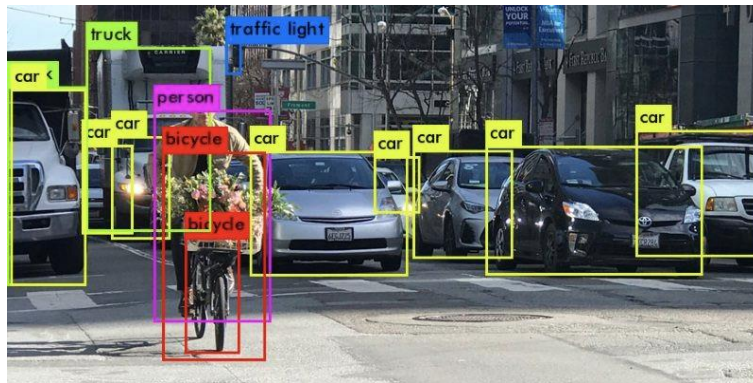
ConvNet Architecture:

- VGG16: 13 convolutional layers and 3 fully connected layers (1000 classes)
- reduces spatial dimensions: 224x224 (image) $\rightarrow \dots \rightarrow 7 \times 7$
- increases depth: 3 (image RGB depth) $\rightarrow \dots \rightarrow 512$ (nr of filters)
- Large network. Even larger version: VGG19

ConvNet Architectures

Darknet-53 (2018) was a breakthrough in real-time object recognition: good recognition results in milliseconds

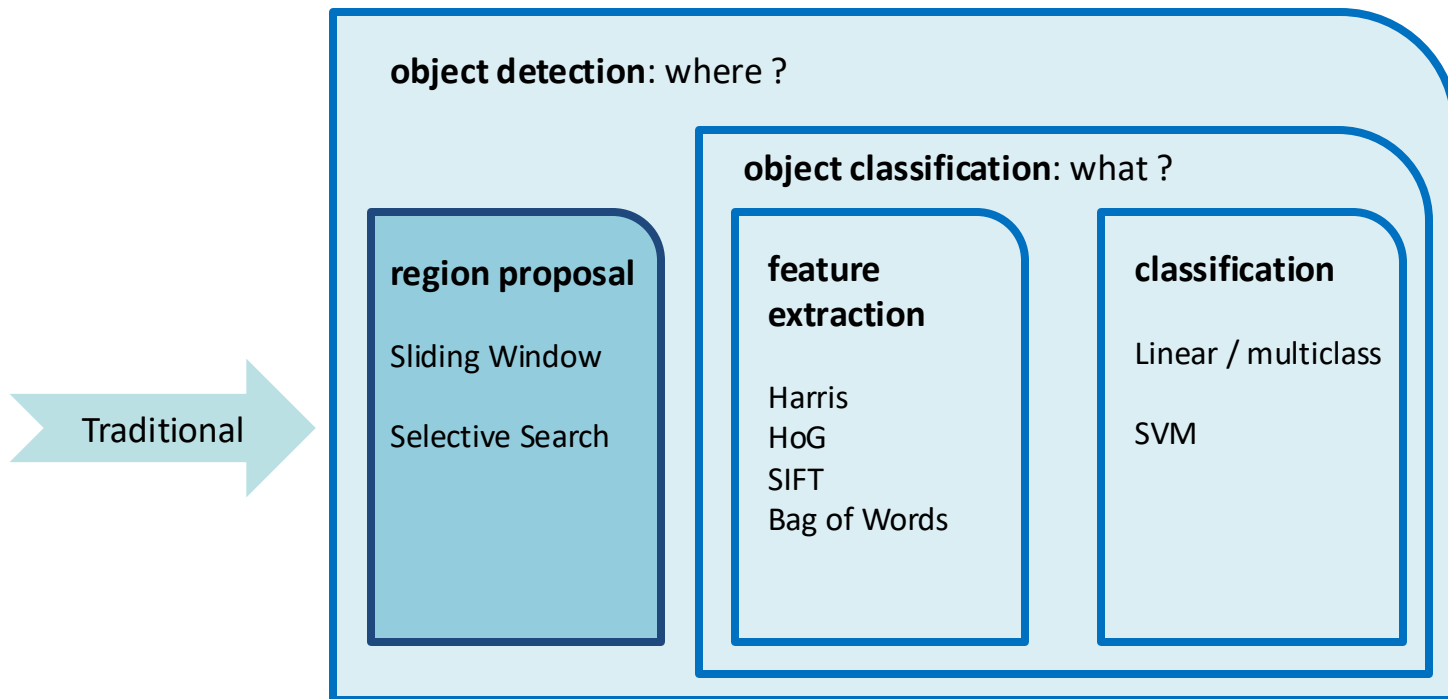
	Type	Filters	Size	Output
1x	Convolutional	32	3 × 3	256 × 256
	Convolutional	64	3 × 3 / 2	128 × 128
	Convolutional	32	1 × 1	
	Convolutional	64	3 × 3	
	Residual			128 × 128
2x	Convolutional	128	3 × 3 / 2	64 × 64
	Convolutional	64	1 × 1	
	Convolutional	128	3 × 3	
	Residual			64 × 64
8x	Convolutional	256	3 × 3 / 2	32 × 32
	Convolutional	128	1 × 1	
	Convolutional	256	3 × 3	
	Residual			32 × 32
8x	Convolutional	512	3 × 3 / 2	16 × 16
	Convolutional	256	1 × 1	
	Convolutional	512	3 × 3	
	Residual			16 × 16
4x	Convolutional	1024	3 × 3 / 2	8 × 8
	Convolutional	512	1 × 1	
	Convolutional	1024	3 × 3	
	Residual			8 × 8
	Avgpool		Global	
	Connected		1000	
	Softmax			

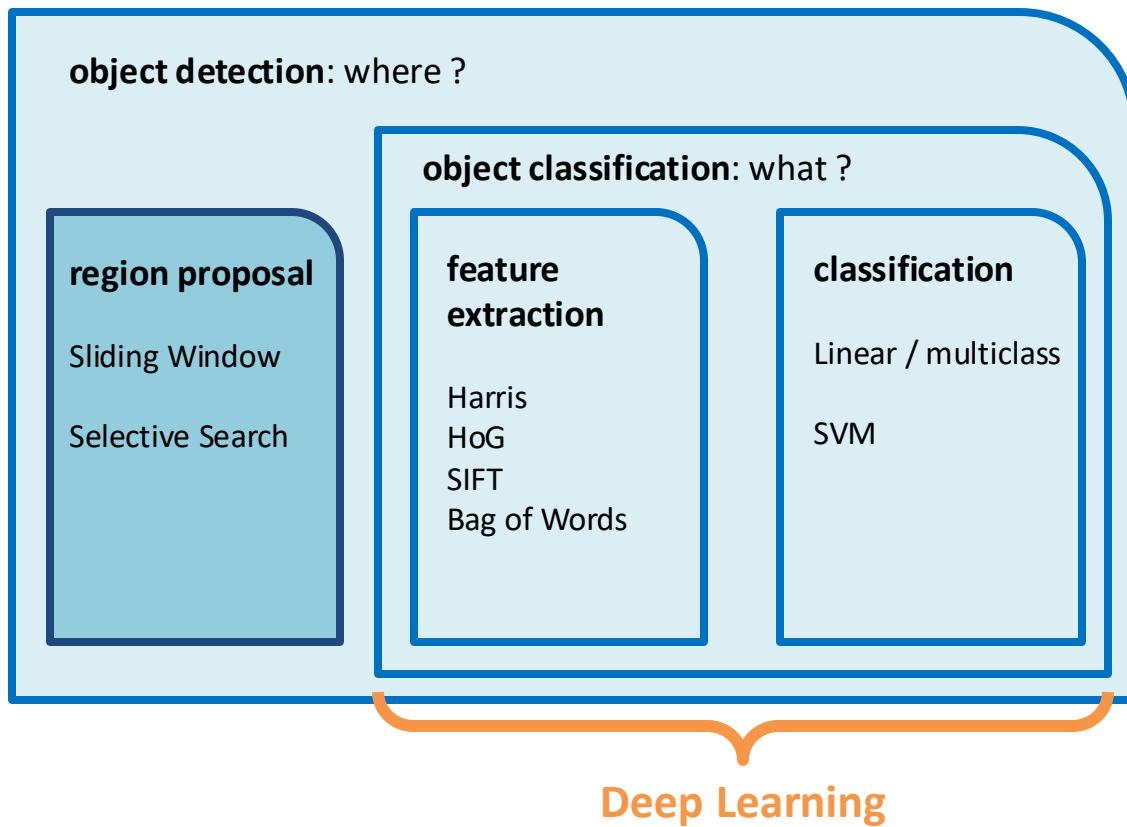


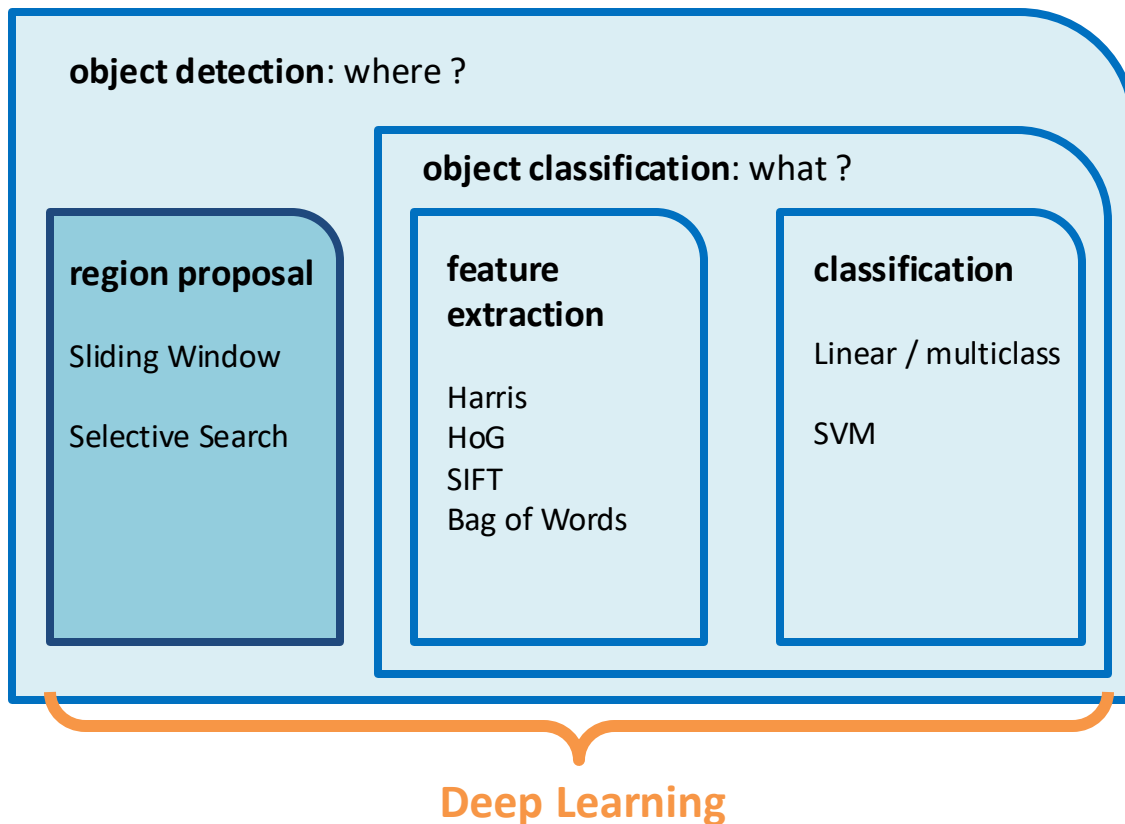
Today's Agenda



- Convolutional Neural Networks
- **Object Detection with Region-based CNNs**

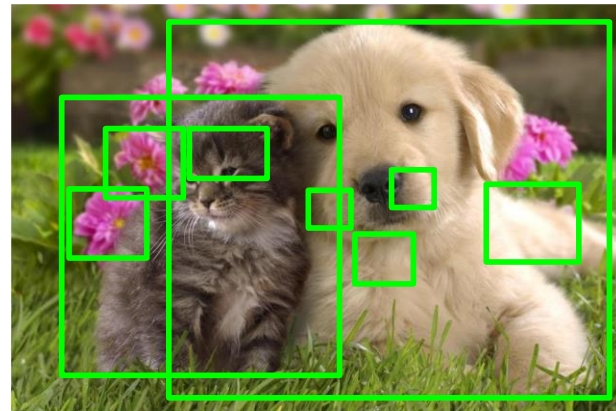






Region Proposals

- Find a small set of boxes that are likely to cover all objects
- Often based on heuristics: e.g. look for “blob-like” image regions
- Relatively fast to run; e.g. Selective Search gives 2000 region proposals in a few seconds on CPU



Alexe et al, “Measuring the objectness of image windows”, TPAMI 2012

Uijlings et al, “Selective Search for Object Recognition”, IJCV 2013

Cheng et al, “BING: Binarized normed gradients for objectness estimation at 300fps”, CVPR 2014

Zitnick and Dollar, “Edge boxes: Locating object proposals from edges”, ECCV 2014

Region-based CNN (R-CNN)

Input
image



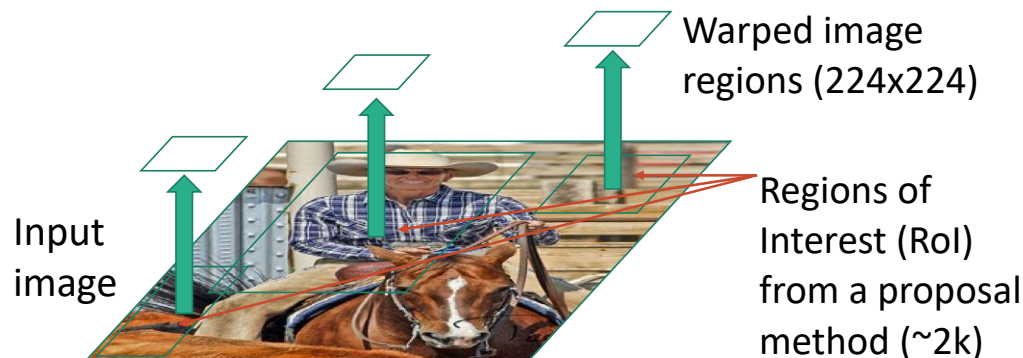
Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

Region-based CNN (R-CNN)



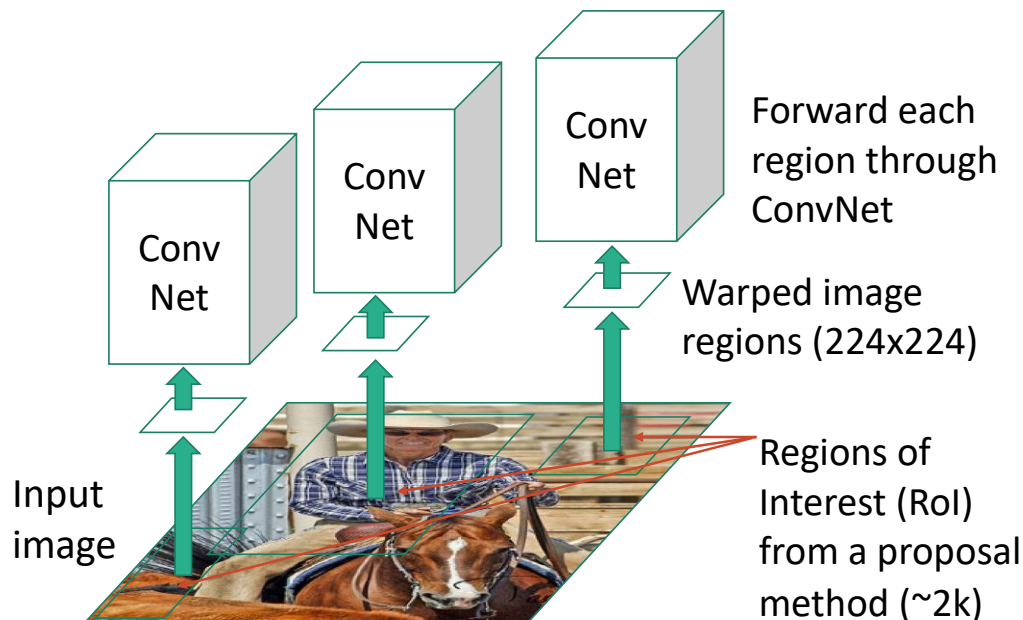
Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

Region-based CNN (R-CNN)



Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

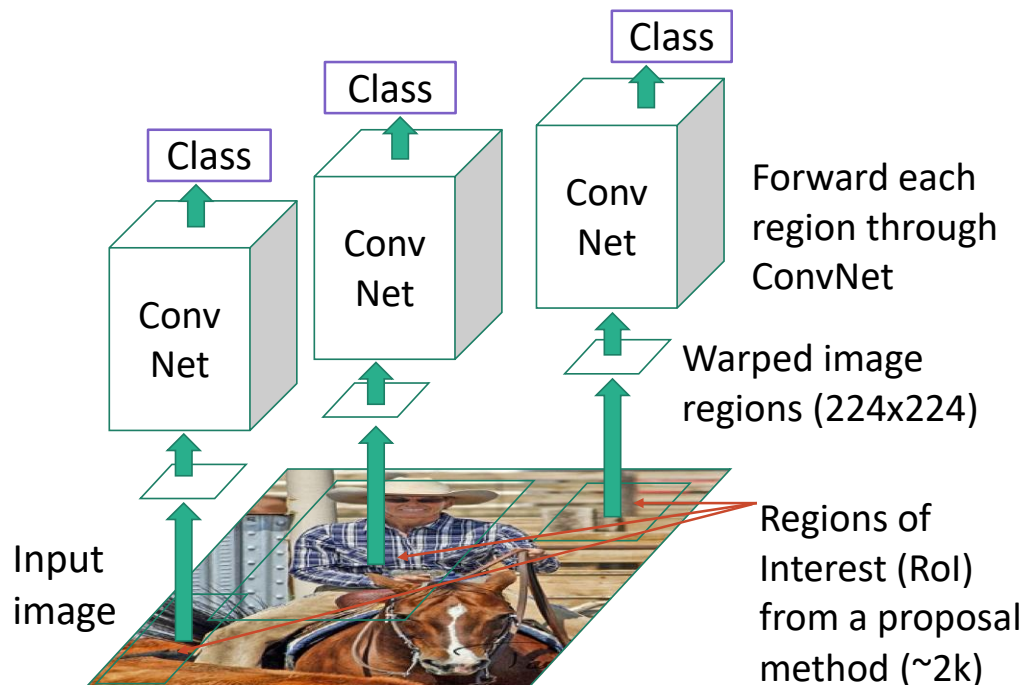
Region-based CNN (R-CNN)



Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

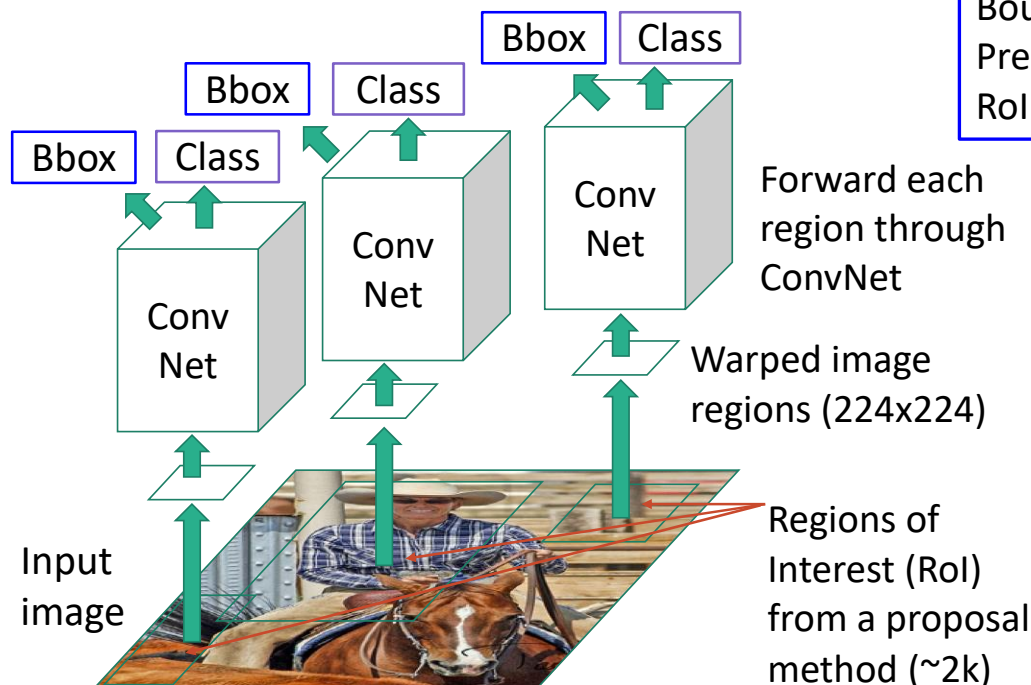
Region-based CNN (R-CNN)

Classify each region



Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

Region-based CNN (R-CNN)

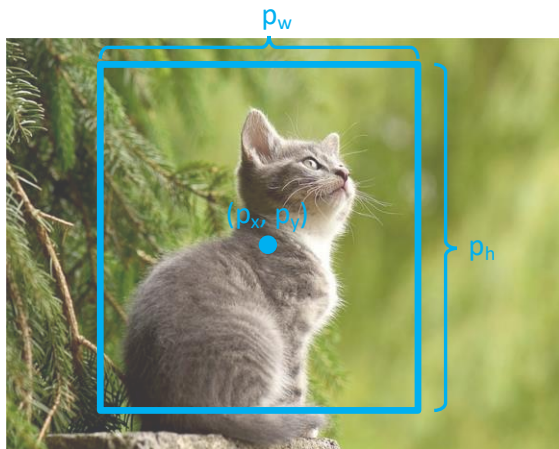


Classify each region

Bounding box regression:
Predict "transform" to correct the
RoI: 4 numbers (t_x, t_y, t_h, t_w)

Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

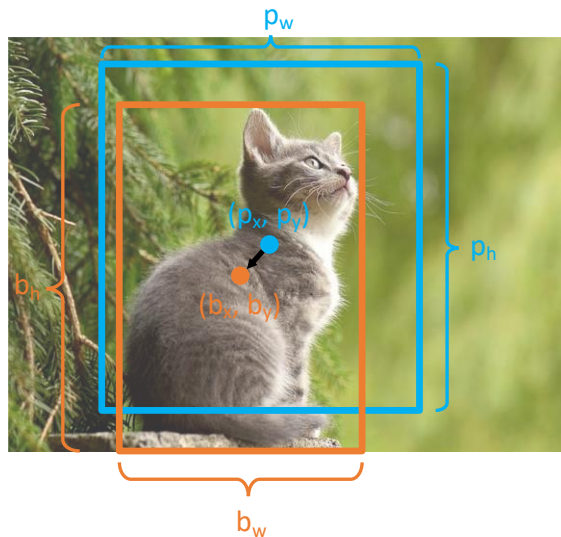
R-CNN: Box Regression



Consider a **region proposal** with center (x_c, y_c) , width w , height h

Model predicts a **transform** (t_x, t_y, t_s, t_c) to correct the region proposal

R-CNN: Box Regression



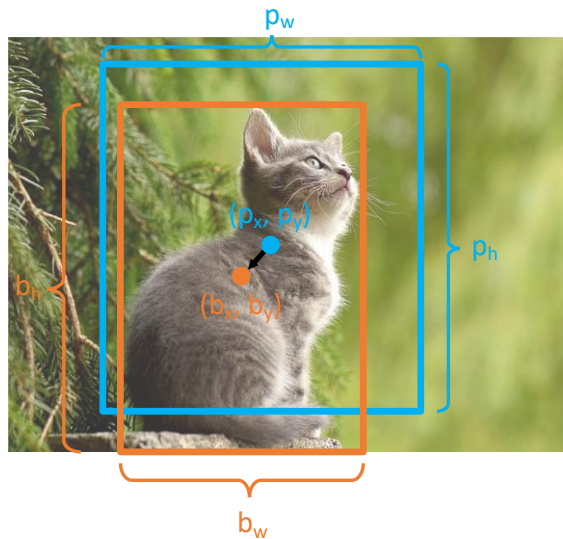
Consider a **region proposal** with center (p_x, p_y) , width p_w , height p_h

Model predicts a **transform** (t_x, t_y, t_w, t_h) to correct the region proposal

The **output box** is defined by:

$$\begin{aligned}
 t_x &= p_x + p_w t_x && \text{Shift center by amount} \\
 t_y &= p_y + p_h t_y && \text{relative to proposal size} \\
 t_w &= p_w \exp(t_w) && \text{Scale proposal; exp ensures} \\
 t_h &= p_h \exp(t_h) && \text{that scaling factor is } > 0
 \end{aligned}$$

R-CNN: Box Regression



Consider a **region proposal** with center (p_x, p_y) , width p_w , height p_h

Model predicts a **transform** (t_x, t_y, t_s, t_c) to correct the region proposal

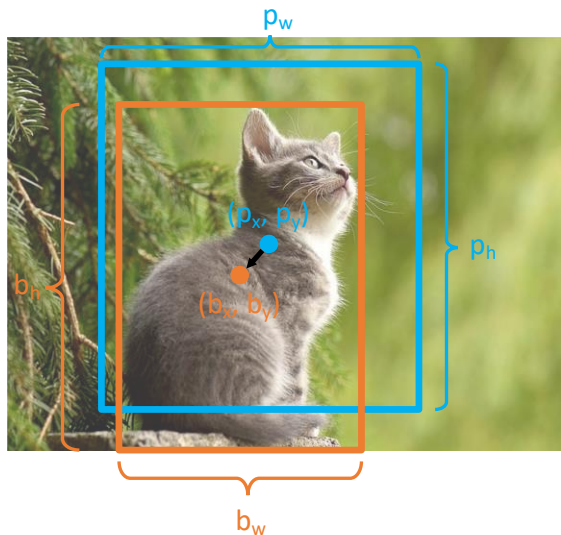
The **output box** is:

$$\begin{aligned} b_x &= p_x + p_w t_x \\ b_y &= p_y + p_h t_y \\ b_w &= p_w \exp(t_s) \\ b_h &= p_h \exp(t_c) \end{aligned}$$

When transform is 0, output = proposal

L2 regularization encourages leaving proposal unchanged

R-CNN: Box Regression



Consider a **region proposal** with center (p_x, p_y) , width p_w , height p_h

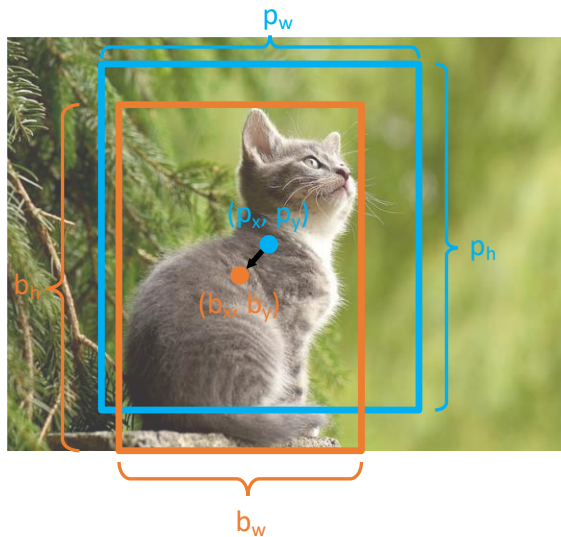
Model predicts a **transform** (t_x, t_y, t_w, t_h) to correct the region proposal

The **output box** is:

$$\begin{aligned} b_x &= p_x + p_w t_x \\ b_y &= p_y + p_h t_y \\ b_w &= p_w \exp(t_w) \\ b_h &= p_h \exp(t_h) \end{aligned}$$

Scale / Translation invariance:
Transform encodes *relative* difference between proposal and output; important since CNN doesn't see absolute size or position after cropping

R-CNN: Box Regression



Consider a **region proposal** with center (p_x, p_y) , width p_w , height p_h

Model predicts a **transform** (t_x, t_y, t_w, t_h) to correct the region proposal

Given **proposal** and **target output**, we can solve for the **transform** the network should output:

The **output box** is:

$$b_x = p_x + p_w t_x$$

$$b_y = p_y + p_h t_y$$

$$b_w = p_w \exp(t_w)$$

$$b_h = p_h \exp(t_h)$$

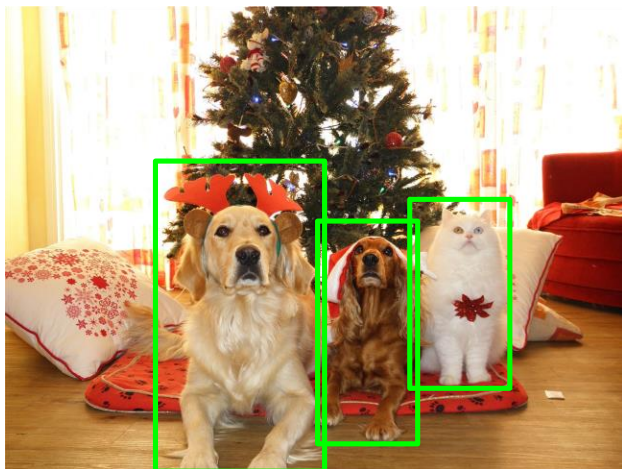
$$t_x = (b_x - p_x) / p_w$$

$$t_y = (b_y - p_y) / p_h$$

$$t_w = \log(b_w / p_w)$$

$$t_h = \log(b_h / p_h)$$

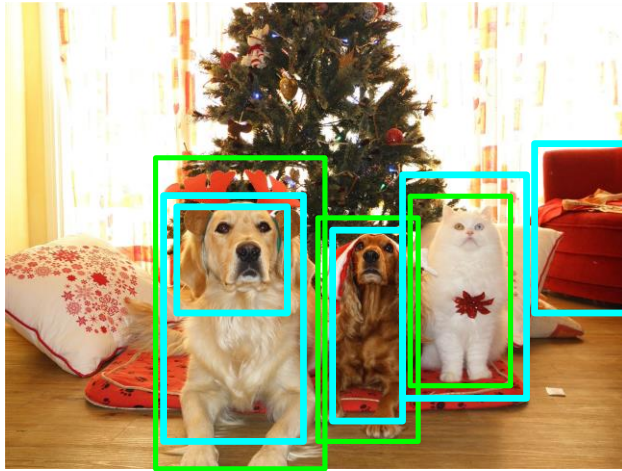
Input Image



Ground-Truth boxes

R-CNN Training

Input Image

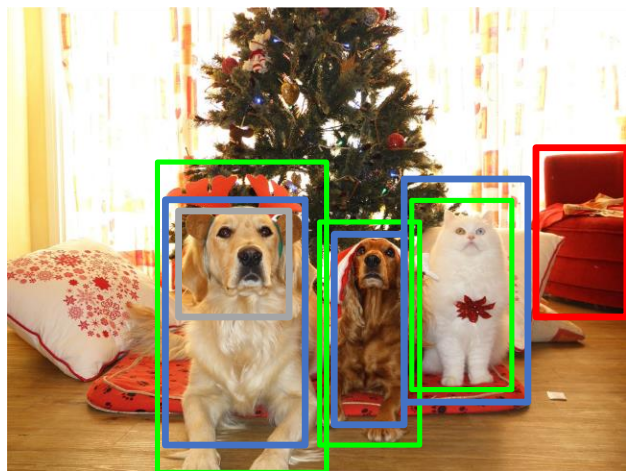


Ground-Truth boxes

Region Proposals

R-CNN Training

Input Image



GT Boxes

Positive

Neutral

Negative

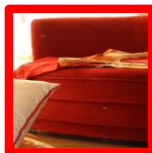
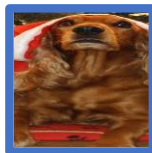
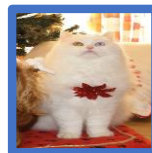
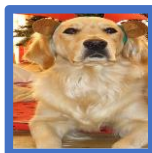
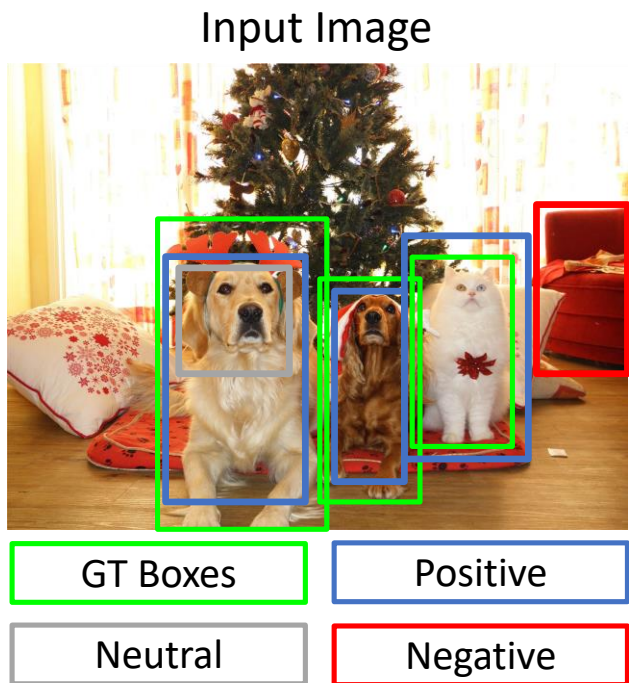
Categorize each region proposal as **positive**, **negative**, or **neutral** based on overlap with ground-truth boxes:

Positive: > 0.5 IoU with a GT box

Negative: < 0.3 IoU with all GT boxes

Neutral: between 0.3 and 0.5 IoU with GT boxes

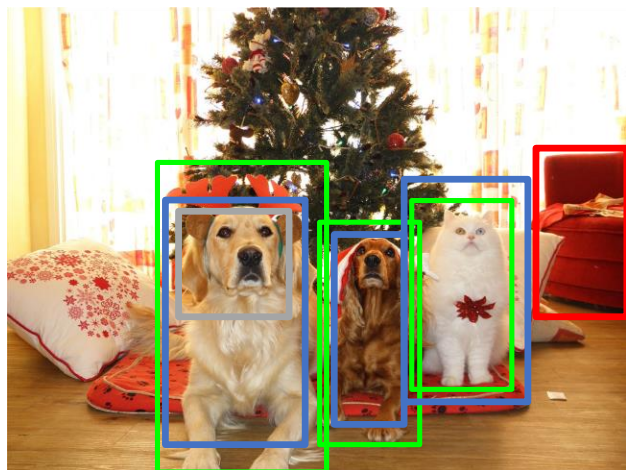
R-CNN Training



Crop pixels from
each positive and
negative proposal,
resize to 224 x 224

R-CNN Training

Input Image



GT Boxes

Positive

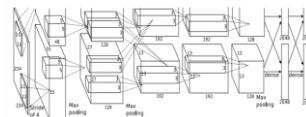
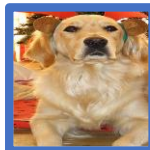
Neutral

Negative

Run each region through CNN

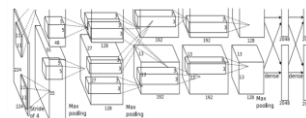
Positive regions: predict class and transform

Negative regions: just predict class



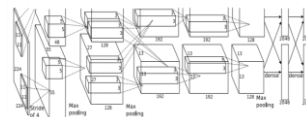
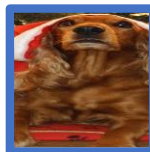
Class target: Dog

Box target: →



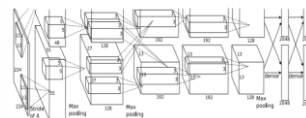
Class target: Cat

Box target: →



Class target: Dog

Box target: →

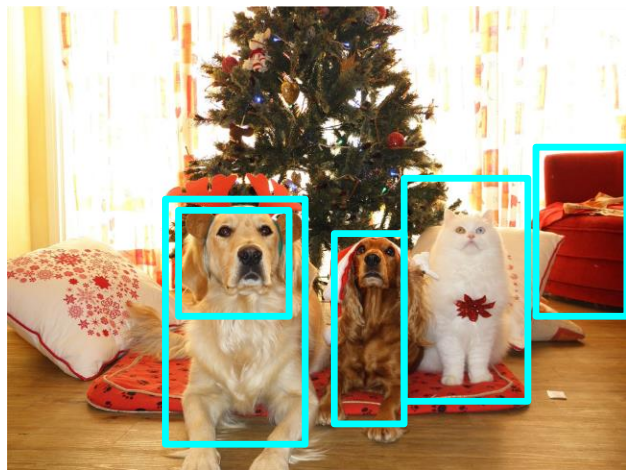


Class target: Background

Box target: None

R-CNN Test-Time

Input Image



Region Proposals

1. Run proposal method
2. Run CNN on each proposal to get class scores, transforms
3. Threshold class scores to get a set of detections

2 problems:

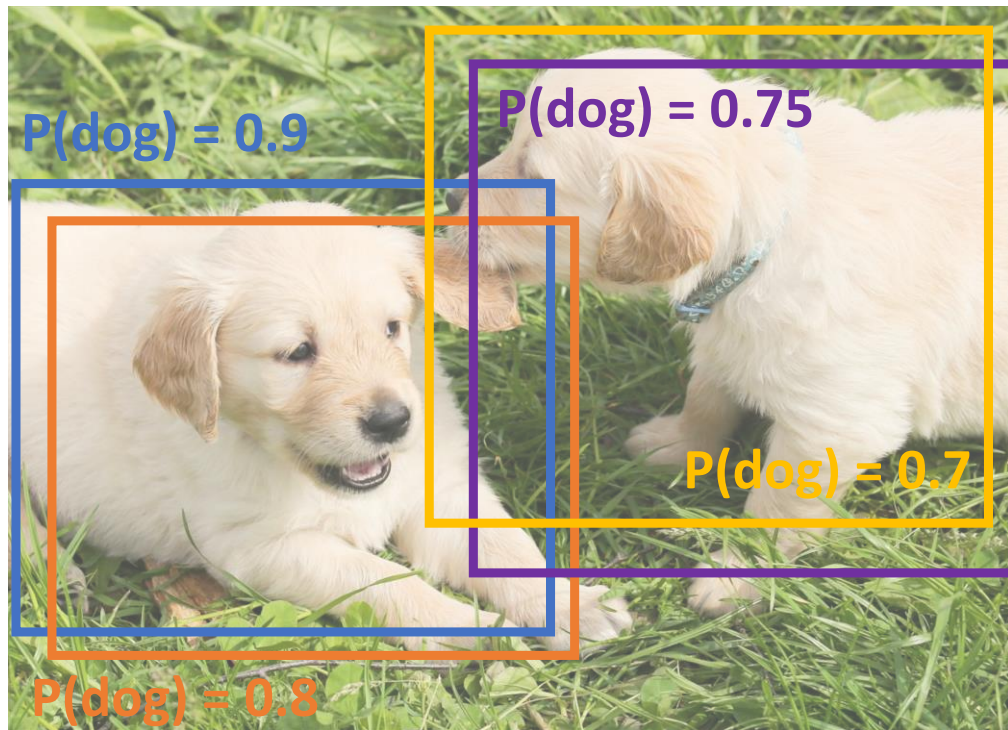
- CNN often outputs overlapping boxes
- How to set thresholds?

Filter Boxes: Non-Maximum Suppression (NMS)

Problem: Object detectors often output many overlapping detections

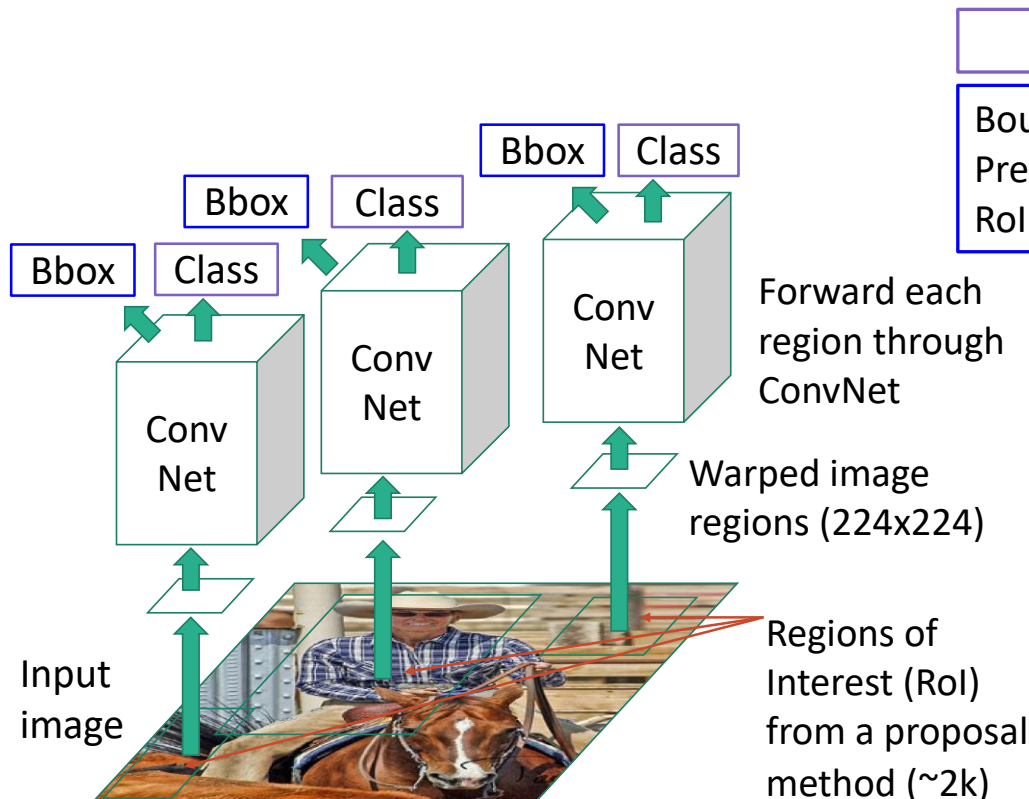
Solution: Post-process raw detections using **Non-Max Suppression (NMS)**:

1. Select next highest-scoring box
2. Eliminate lower-scoring boxes with $\text{IoU} > \text{threshold}$ (e.g. 0.7)
3. If any boxes remain, GOTO 1



[Puppy image is [CC0 Public Domain](#)]

R-CNN - Drawbacks

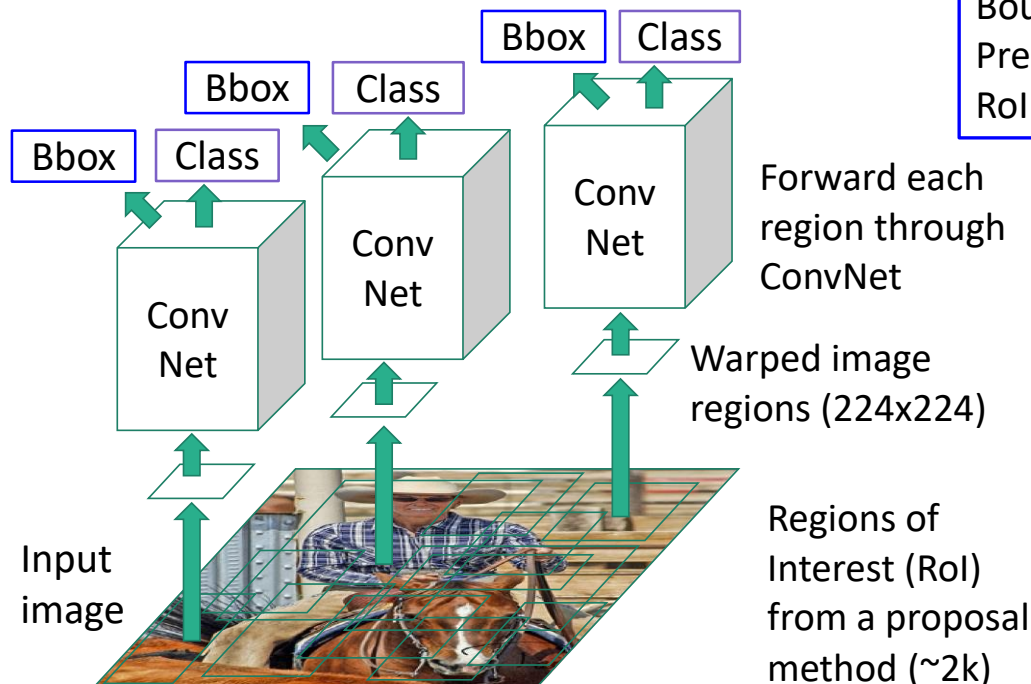


Classify each region

Bounding box regression:
Predict "transform" to correct the
RoI: 4 numbers (t_x , t_y , t_h , t_w)

Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

R-CNN - Drawbacks



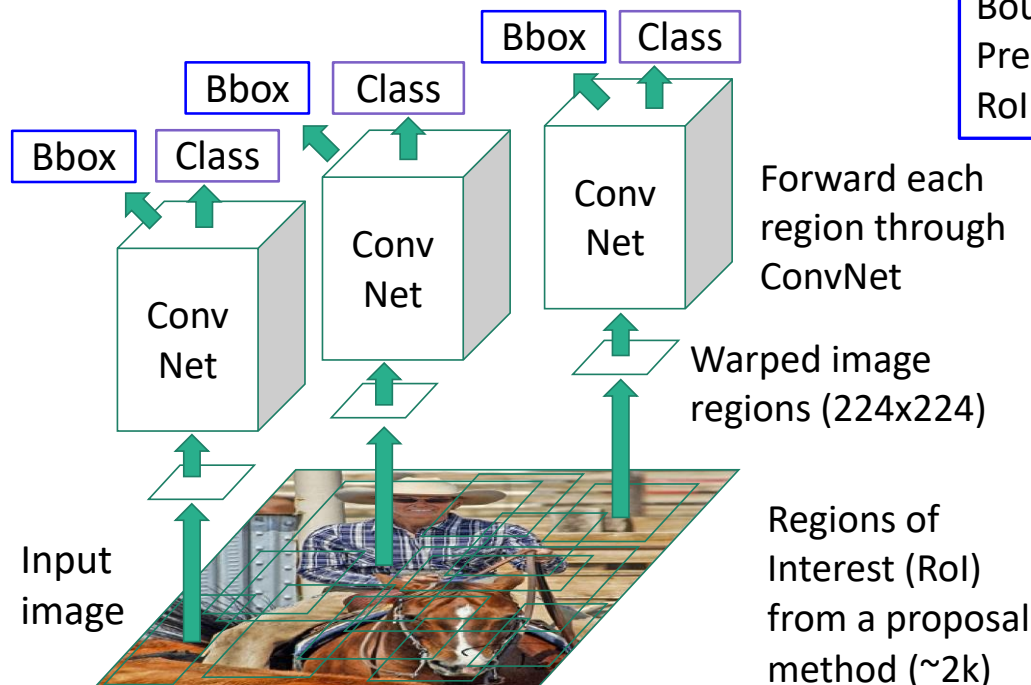
Classify each region

Bounding box regression:
Predict "transform" to correct the
RoI: 4 numbers (t_x, t_y, t_h, t_w)

**Problem: Very slow! Need
to do 2000 forward passes
through CNN per image**

Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

R-CNN - Drawbacks



Classify each region

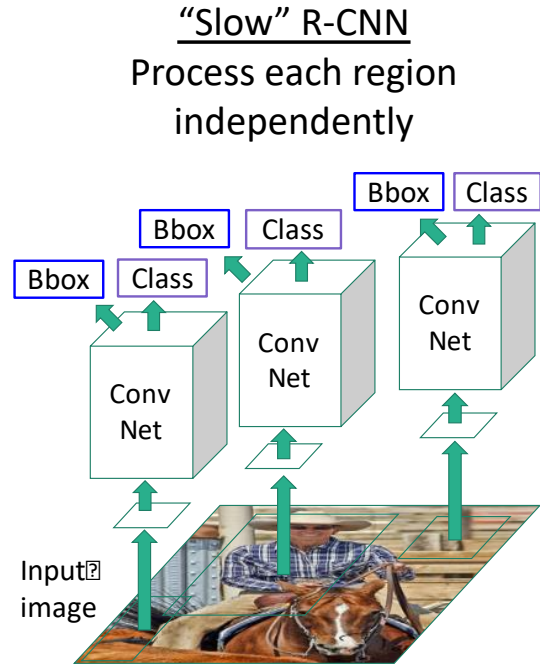
Bounding box regression:
Predict "transform" to correct the
RoI: 4 numbers (t_x, t_y, t_h, t_w)

Problem: Very slow! Need to do 2000 forward passes through CNN per image

Idea: Overlapping proposals cause a lot of repeated work: same pixels processed many times. Can we avoid this?

Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

R-CNN - Drawbacks



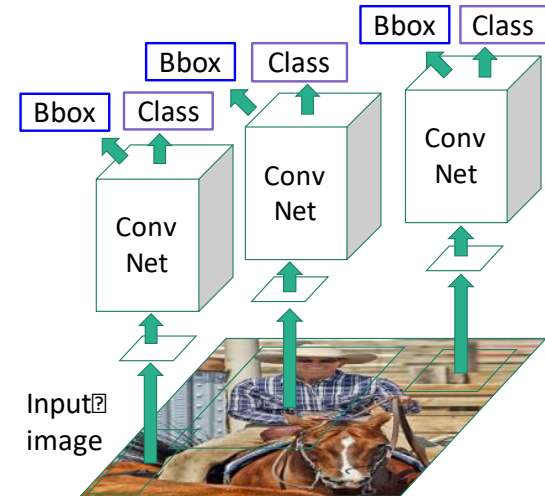
R-CNN - Drawbacks



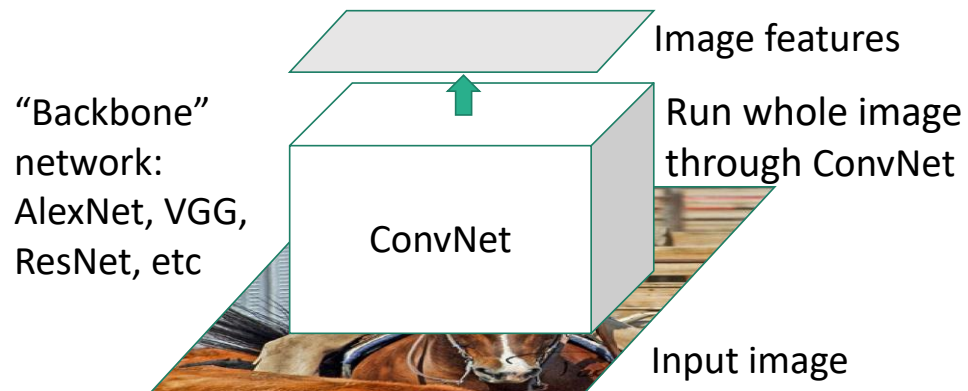
Input image

Girshick, "Fast R-CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

"Slow" R-CNN
Process each region
independently

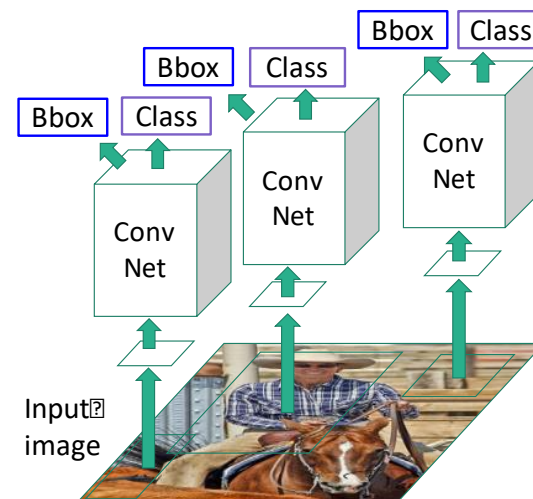


R-CNN - Drawbacks



Girshick, “Fast R-CNN”, ICCV 2015. Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

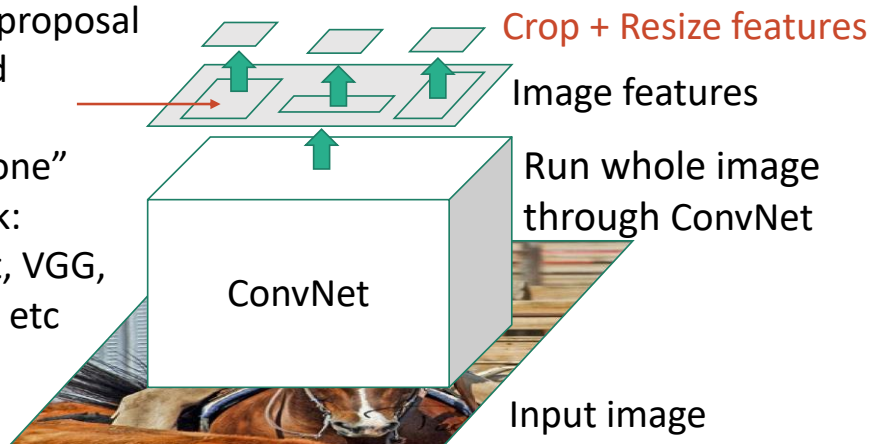
“Slow” R-CNN
Process each region
independently



R-CNN - Drawbacks

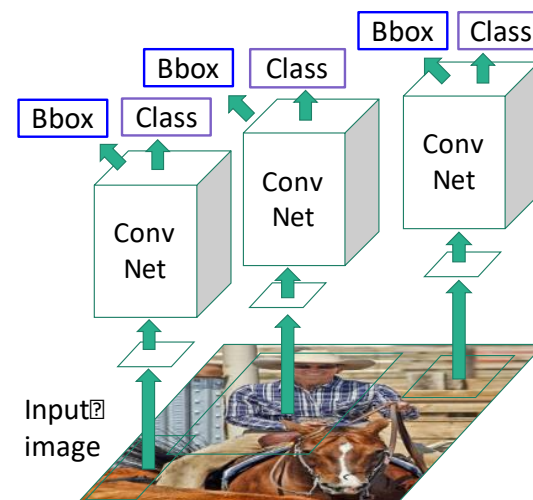
Regions of Interest (Rois) from a proposal method

“Backbone” network:
AlexNet, VGG,
ResNet, etc



Girshick, “Fast R-CNN”, ICCV 2015. Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

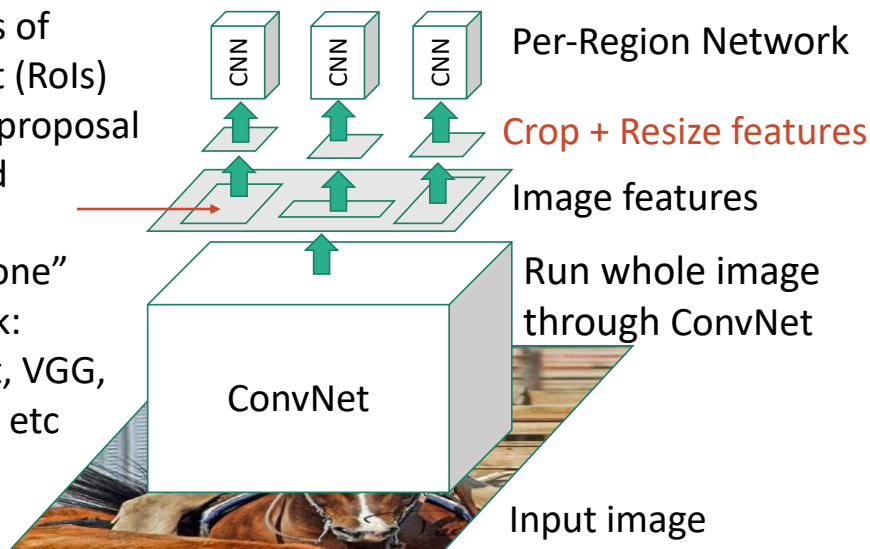
“Slow” R-CNN
Process each region independently



Fast R-CNN

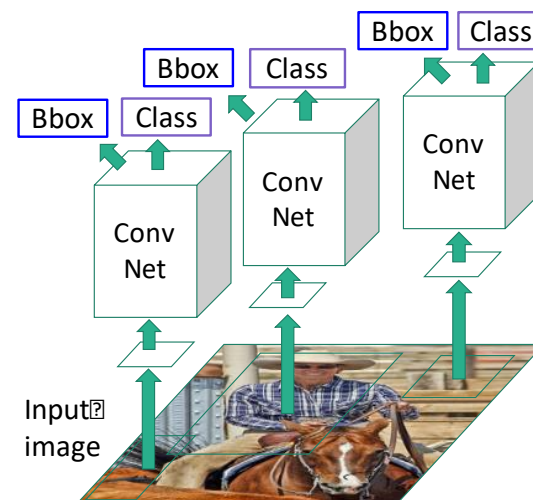
Regions of Interest (RoIs) from a proposal method

“Backbone” network:
AlexNet, VGG, ResNet, etc

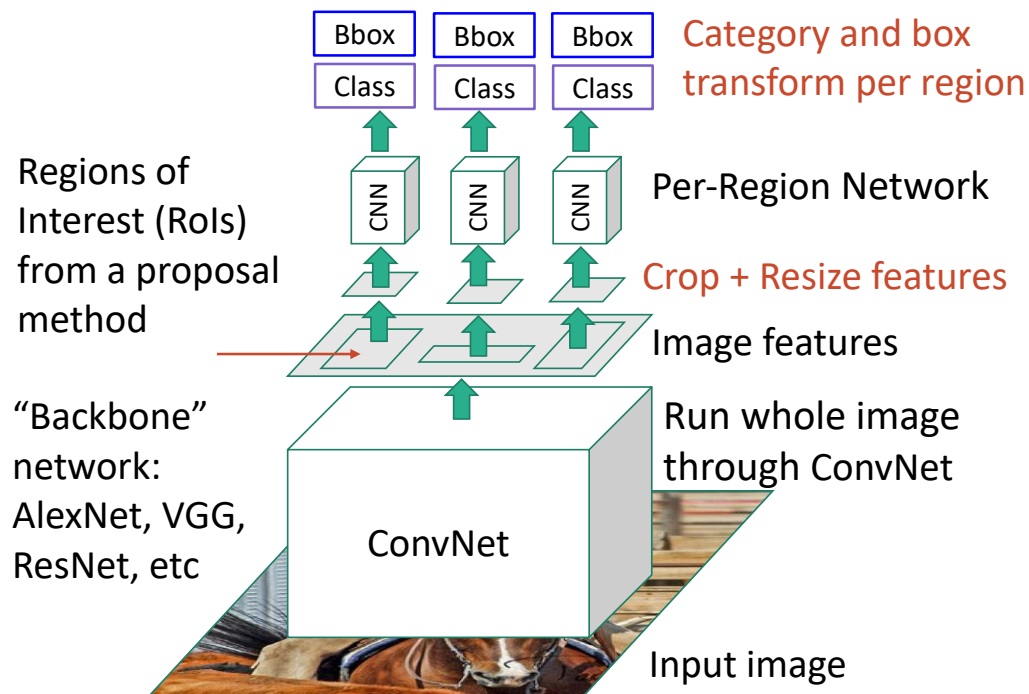


Girshick, “Fast R-CNN”, ICCV 2015. Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

“Slow” R-CNN
Process each region independently

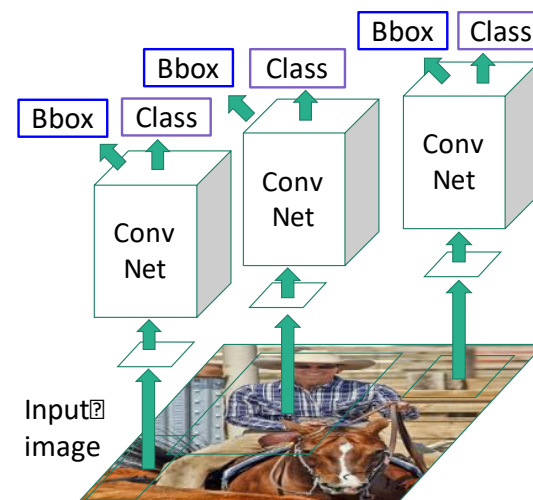


Fast R-CNN



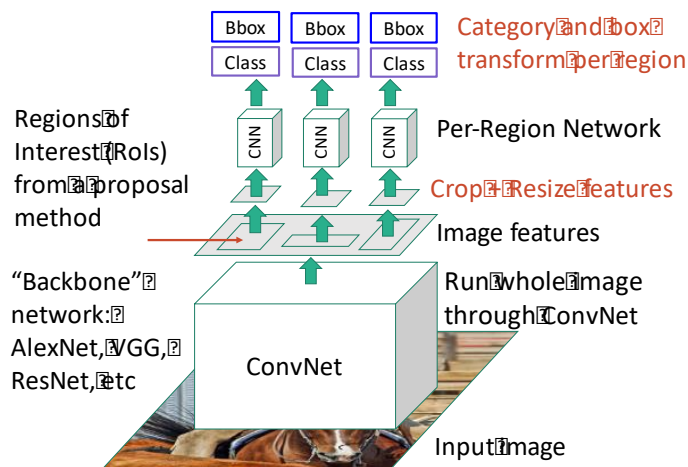
Girshick, "Fast R-CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

"Slow" R-CNN
Process each region independently

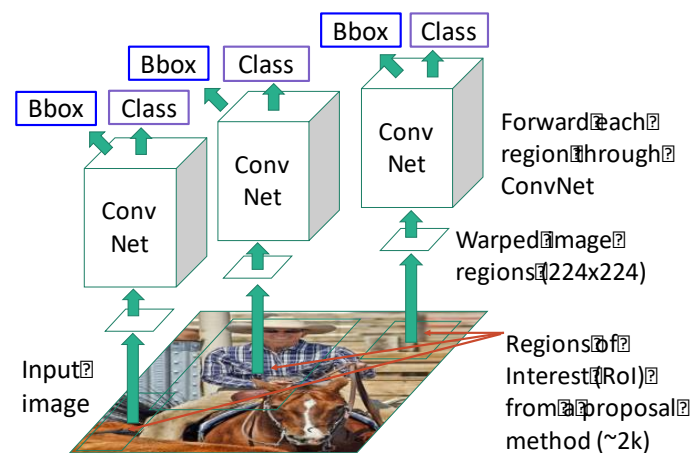


Fast R-CNN vs. (Slow) R-CNN

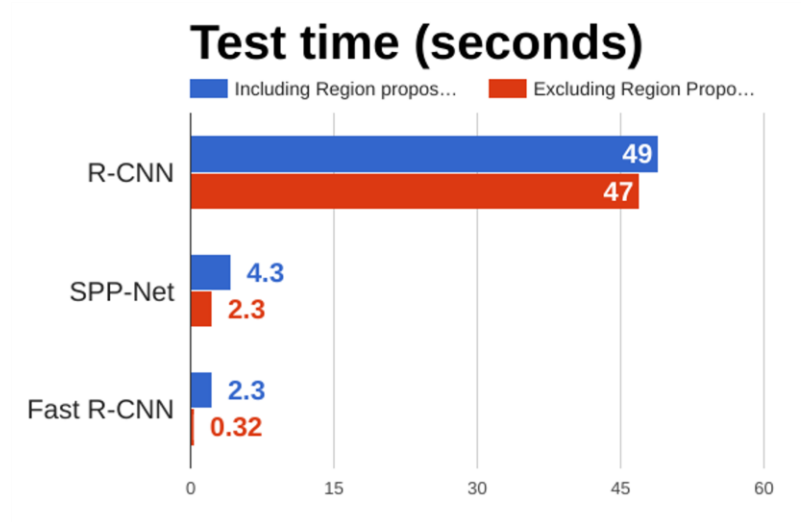
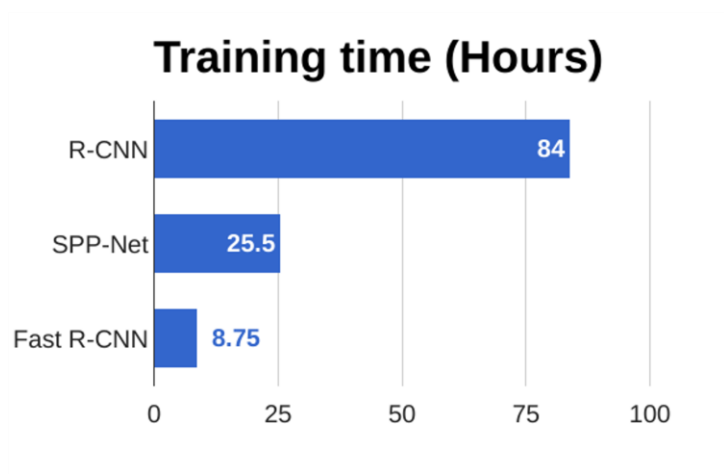
Fast R-CNN: Apply differentiable cropping to shared image features



"Slow" R-CNN: Apply differentiable cropping to shared image features



Fast R-CNN vs. (Slow) R-CNN

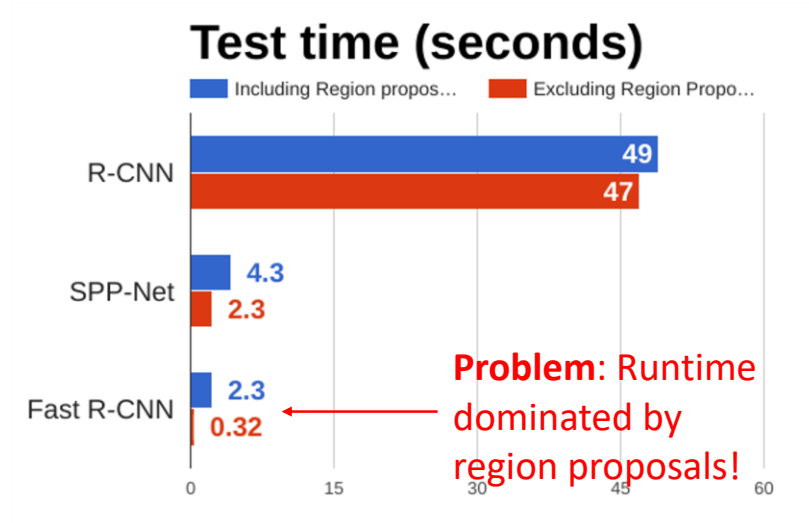
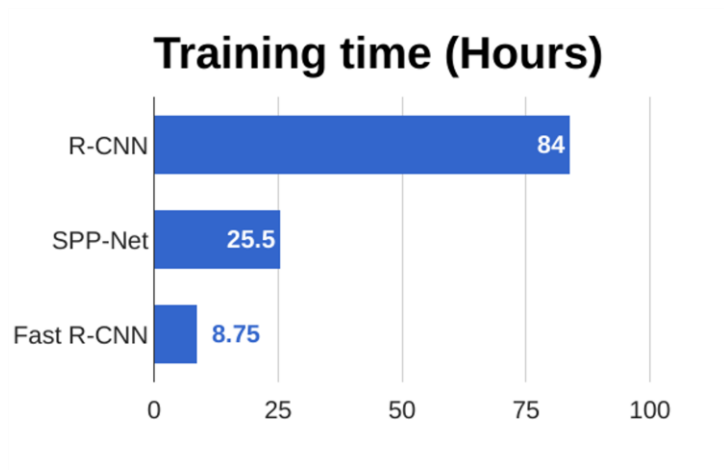


Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.

He et al, "Spatial pyramid pooling in deep convolutional networks for visual recognition", ECCV 2014

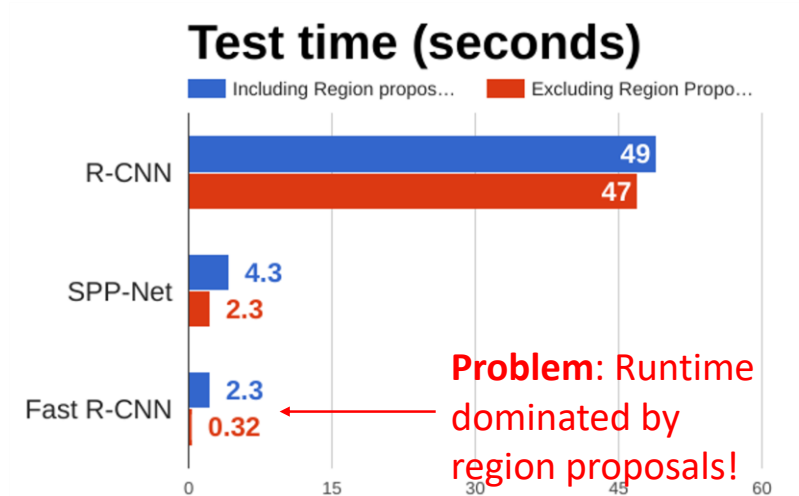
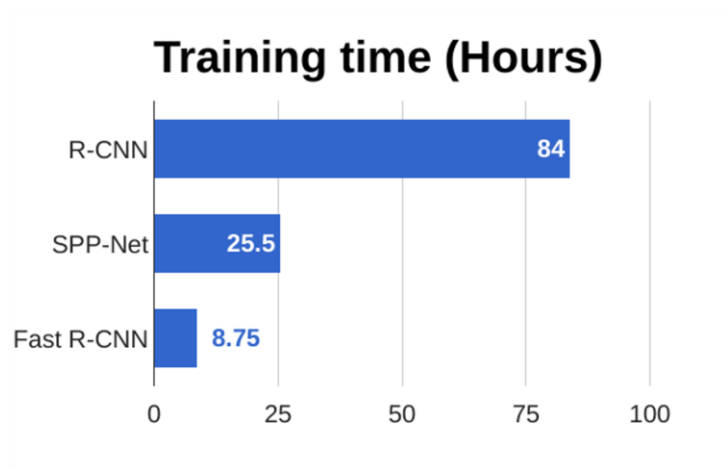
Girshick, "Fast R-CNN", ICCV 2015

Fast R-CNN vs. (Slow) R-CNN



Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
He et al, "Spatial pyramid pooling in deep convolutional networks for visual recognition", ECCV 2014
Girshick, "Fast R-CNN", ICCV 2015

Fast R-CNN vs. (Slow) R-CNN



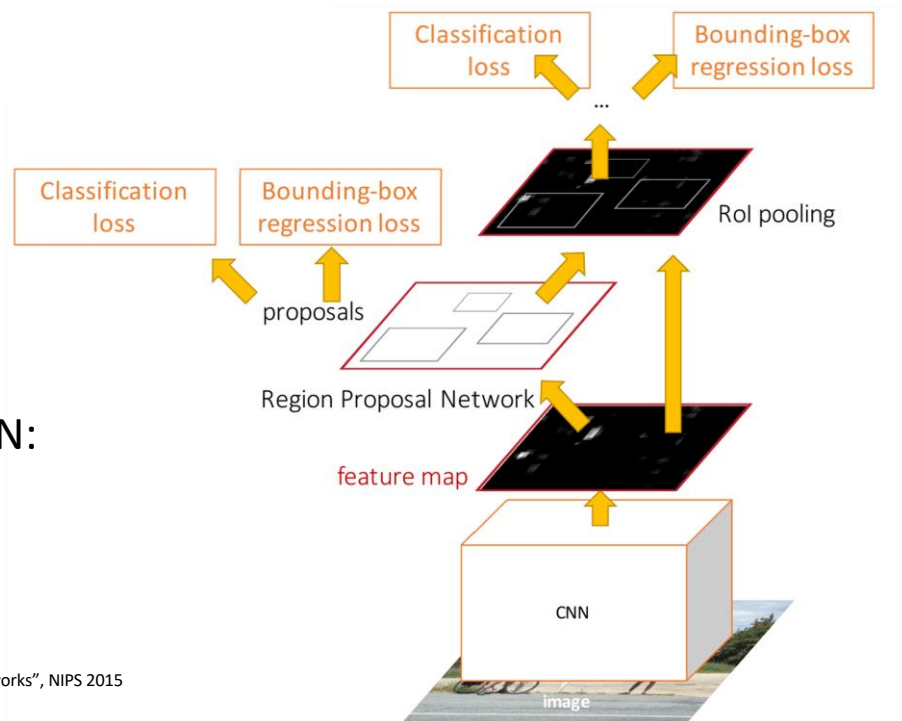
Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
He et al, "Spatial pyramid pooling in deep convolutional networks for visual recognition", ECCV 2014
Girshick, "Fast R-CNN", ICCV 2015

Recall: Region proposals computed by heuristic "Selective Search" algorithm on CPU -- let's learn them with a CNN instead!

Faster R-CNN: Learnable Region Proposals

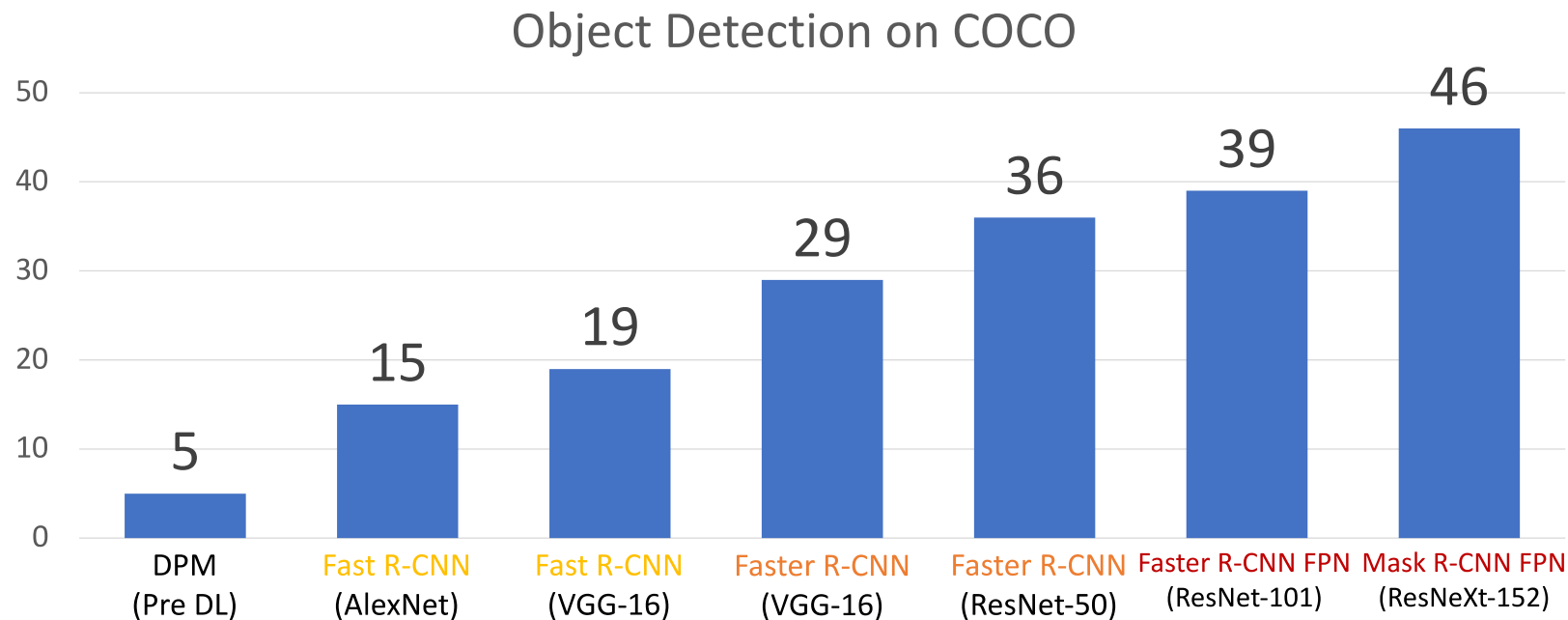
Insert **Region Proposal Network (RPN)** to predict proposals from features

Otherwise same as Fast R-CNN:
Crop features for each proposal, classify each one



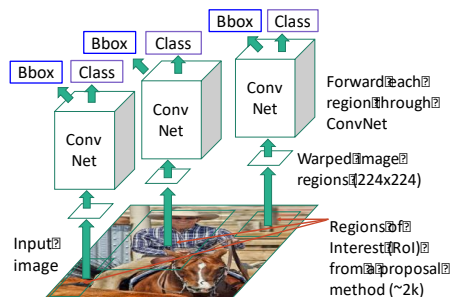
Ren et al, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks", NIPS 2015
Figure copyright 2015, Ross Girshick; reproduced with permission

Overview: R-CNN Variants

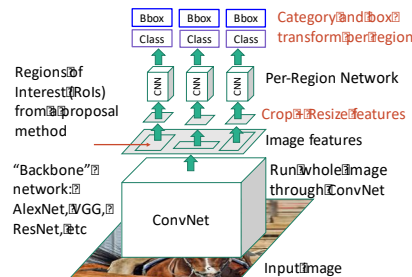


Overview: R-CNN Variants

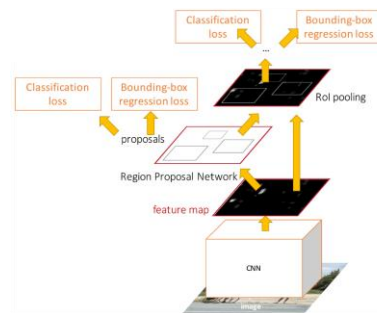
“Slow” R-CNN: Run CNN independently for each region



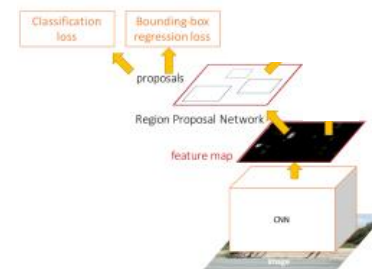
Fast R-CNN: Apply differentiable cropping to shared image features



Faster R-CNN: Compute proposals with CNN



Single-Stage: Fully convolutional detector



With anchors: RetinaNet
Anchor-Free: FCOS

Disclaimer

Many of the slides used here are obtained from online resources (including many open lecture materials) without appropriate acknowledgement. They are used here for the sole purpose of classroom teaching. All the credit and all the copyrights belong to the original authors. You should not copy it, redistribute it, put it online, or use it for any other purposes than for this course.